MODUL PRAKTIKUM

Artificial Intelligence and Its Application

# AI Chatbot dengan RBAC

Modul Praktikum – Chatbot berbasis Role-Based Access Control

| | |
|---|---|
| **Mata Kuliah** | Kecerdasan Artifisial dan Penerapannya |
| **Semester** | Ganjil 2026–2027 |
| **Dosen Pengampu** | 1. Mochamad Nizar Palefi Ma'ady, S.Kom., M.Kom., M.IM |
| **Tim Asisten Praktikum** | 1. Muhammad Baihaqi<br>2. Mohamad Fikri Isfahani |

# Custom AI Chatbot RAG Berbasis Role dengan FastAPI dan Groq

## 1. Pendahuluan

Modul ini membahas perancangan dan pembangunan aplikasi **Custom AI Chatbot RAG Berbasis Role** menggunakan **Python FastAPI**, **SQLite**, **PyJWT**, dan **Groq Cloud LLM**. Pada praktikum di kelas, mahasiswa membangun sistem secara utuh di lingkungan lokal (`localhost`), mencakup panel *Admin* untuk mengelola dokumen pengetahuan (PDF, DOCX, XLSX, CSV, TXT), mengatur status aktif/nonaktif dokumen, memantau potongan teks (*chunks*), serta memantau kuota Groq API, dan halaman *Chatbot Mahasiswa* yang dapat diakses langsung tanpa login.

Berbeda dengan chatbot umum, sistem ini menerapkan **Role-Based Access Control (RBAC)** pada tingkat basis data dokumen. Setiap dokumen yang diunggah dipetakan ke satu atau beberapa *role* tertentu melalui tabel relasi `document_roles`. Saat pengguna mengajukan pertanyaan, mesin RAG hanya mengambil *chunks* dari dokumen yang aktif dan sesuai dengan *role* pengguna tersebut, lalu menyertakan sumber referensi yang dapat diklik untuk melihat isi dokumen aslinya.

### Arsitektur Sistem

| No | Komponen | Teknologi | Fungsi |
|---|---|---|---|
| 1 | Backend API Server | FastAPI & Uvicorn | Menyajikan halaman web, REST API autentikasi admin, manajemen role & dokumen, serta endpoint chat RAG. |
| 2 | Database & Auth | SQLite & PyJWT | Menyimpan data admin, role, metadata dokumen, vector chunks, log kuota Groq, serta validasi token JWT. |
| 3 | Ekstraktor Dokumen | pypdf, python-docx, CSV/XLSX | Mengekstrak teks dan tabel Markdown dari file PDF, DOCX, XLSX, CSV, dan TXT secara otomatis. |
| 4 | Mesin RAG & LLM | app/rag.py & Groq SDK | Melakukan chunking, embedding hashing + cosine similarity + lexical bonus, serta generasi jawaban dengan Groq. |
| 5 | Antarmuka Web | HTML5, Tailwind CSS, Vanilla JS | Menyediakan halaman Chat Mahasiswa (tanpa login) dan Dashboard Admin dengan popup chunks, chatbot, dan kuota. |

Ketika pengguna mengirim pertanyaan melalui antarmuka web, backend memprosesnya secara terstruktur:

**Alur Pemrosesan Pertanyaan (RAG Berbasis Role)**

```
Pertanyaan Pengguna + Role (misal: "mahasiswa")
    │
    ├─ 1. Validasi Role pada tabel `roles`
    │
    ├─ 2. Pencarian Hybrid (`search_chunks_by_role`):
    │      • Filter hanya dokumen dengan `is_active = 1` dan terhubung ke `role` terkait
    │      • Hitung Cosine Similarity (embedding 384-dimensi) + Lexical Bonus (kata kunci
& frasa)
```

```
    │       • Ambil Top-K chunks dengan skor >= min_score (0.16)
    │
    ├─ 3. Cek Sapaan / Fallback:
    │       • Jika tidak ada chunk relevan & berupa sapaan -> balas ramah tanpa memanggil
LLM
    │       • Jika tidak ada chunk relevan -> balas sopan bahwa informasi belum tersedia
    │
    └─ 4. Generasi Jawaban (`generate_answer`):
            • Gabungkan konteks Top-K chunks + System Prompt konsistensi
            • Kirim ke Groq LLM (`qwen/qwen3.8-27b`) + catat token & rate-limit ke
`groq_usage_logs`
            • Kembalikan jawaban bersih + daftar sumber referensi ke frontend
```

**Mengapa RAG Berbasis Role?** Dengan memisahkan akses dokumen berdasarkan *role*, satu sistem chatbot dapat melayani berbagai kelompok pengguna (misalnya `mahasiswa` dan `dosen`) secara aman. Dokumen sensitif atau pedoman internal yang hanya ditujukan untuk *role* tertentu tidak akan pernah ikut terbaca saat *role* lain mengajukan pertanyaan.

## Tujuan Pembelajaran

- Membangun backend REST API modern menggunakan **Python FastAPI** dan database relasional **SQLite**.
- Menerapkan autentikasi admin berbasis **JWT (JSON Web Token)** dan *password hashing* PBKDF2-SHA256.
- Mengimplementasikan pipeline **ekstraksi multi-format** (PDF, DOCX, XLSX, CSV, TXT) dan *table-aware chunking*.
- Menerapkan mesin **Retrieval-Augmented Generation (RAG)** berbasis *role* dengan pencarian vektor dan leksikal.
- Mengintegrasikan **Groq Cloud LLM** lengkap dengan fitur pemantauan kuota (RPD/TPM) dan tampilan sumber referensi.

## Prasyarat

1. Install **Python 3.10 atau lebih baru** dari situs resmi `https://www.python.org/` (centang *Add Python to PATH* saat instalasi).
2. Pastikan Python dan pip aktif dengan menjalankan `python --version` dan `pip --version` di terminal.
3. Siapkan teks editor **Visual Studio Code** dan browser modern (Chrome/Edge).
4. Siapkan akun Groq di `https://console.groq.com/` untuk membuat **GROQ_API_KEY**.
5. Unduh dataset default praktikum dari repositori GitHub: https://github.com/Baihaqi2275/modul-praktikum-AI/tree/main/Pertemuan%202%20-%20Custom%20AI%20Chatbot%20RAG%20Berbasis%20Role/dataset.

**Catatan Lingkungan Praktikum:** Selama sesi praktikum di kelas, seluruh aplikasi dijalankan secara lokal pada `http://127.0.0.1:8000`. Pastikan koneksi internet aktif agar backend dapat menghubungi Groq Cloud API dan memuat Tailwind CSS CDN pada antarmuka web.

## 2. Praktikum di Kelas: Membangun Chatbot RAG Berbasis Role

Ikuti langkah-langkah berikut secara berurutan untuk membangun aplikasi Chatbot RAG Berbasis Role dari awal hingga siap diuji.

### Langkah 1: Membuat Folder Project dan Virtual Environment

Buka terminal di Visual Studio Code atau Command Prompt, lalu buat folder project bernama `modul2ai` dan aktifkan *virtual environment* Python agar dependensi project terisolasi rapi.

**Terminal (Windows Command Prompt / PowerShell)**

```
mkdir modul2ai
cd modul2ai
python -m venv venv
venv\Scripts\activate
```

### Langkah 2: Membuat requirements.txt dan Memasang Dependensi

Buat file `requirements.txt` pada folder utama project, lalu pasang seluruh pustaka yang diperlukan menggunakan `pip`.

**requirements.txt**

```
fastapi>=0.115.0
uvicorn>=0.32.0
python-multipart>=0.0.12
python-dotenv>=1.0.1
pypdf>=5.0.0
python-docx>=1.1.2
groq>=0.11.0
PyJWT>=2.9.0
```

**Terminal**

```
pip install -r requirements.txt
```

### Langkah 3: Membuat Struktur Folder dan File

Buat struktur direktori berikut di dalam folder project agar kode backend, frontend, dan dataset tertata modular:

**Struktur Folder Project**

```
modul2ai/
├── app/
│   ├── __init__.py
│   ├── auth.py
│   ├── database.py
│   ├── extractor.py
│   ├── main.py
│   └── rag.py
├── dataset/
│   ├── Alur_Menghubungi_SSC.txt
│   ├── SSC_Operating_Hours.txt
│   ├── SK-TAK-2022.pdf
```

```
│   └── SK TAK TUNC 2025.pdf
├── static/
│   ├── index.html
│   └── admin.html
├── uploads/
├── .env
└── requirements.txt
```

## Langkah 4: Konfigurasi Environment (.env)

Buat file `.env` di root project untuk menyimpan API Key Groq, konfigurasi model LLM, kunci rahasia JWT, serta akun default admin. Ganti nilai `GROQ_API_KEY` dengan API Key milik Anda dari dashboard Groq Console.

**.env**

```
GROQ_API_KEY=gsk_masukkan_api_key_groq_anda_di_sini
GROQ_MODEL=qwen/qwen3.8-27b
JWT_SECRET=modul2ai-super-secret-jwt-key-2026
ADMIN_USERNAME=admin
ADMIN_PASSWORD=admin123
DB_PATH=modul2ai.db
```

## Langkah 5: Menyiapkan Dataset Default (Folder dataset/)

Siapkan dokumen referensi layanan akademik pada folder `dataset/` (misalnya file TXT panduan layanan SSC dan file PDF pedoman TAK). Saat server dijalankan pertama kali dan tabel `documents` masih kosong, fungsi `seed_default_documents()` akan otomatis mengekstrak, memotong menjadi *chunks*, dan mengindeks seluruh file tersebut ke database untuk role `mahasiswa`.

Unduh seluruh file dataset default melalui link berikut lalu simpan ke dalam folder `dataset/`: https://github.com/Baihaqi2275/modul-praktikum-AI/tree/main/Pertemuan%202%20-%20Custom%20AI%20Chatbot%20RAG%20Berbasis%20Role/dataset

## Langkah 6: Membuat Modul Autentikasi (app/auth.py)

File `app/auth.py` menangani *hashing* password admin menggunakan algoritma `pbkdf2_hmac` (SHA-256 + random salt) serta pembuatan dan verifikasi token JWT untuk melindungi seluruh endpoint admin.

**app/auth.py**

```
import hashlib
import hmac
import os
import secrets
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from dotenv import load_dotenv

load_dotenv()

JWT_SECRET = os.getenv("JWT_SECRET", "rahasia_modul2_jwt_2026")
```

```
JWT_ALGORITHM = "HS256"
security = HTTPBearer(auto_error=False)


def hash_password(password: str) -> str:
    salt = secrets.token_hex(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt.encode("utf-8"),
100_000)
    return f"{salt}${dk.hex()}"


def verify_password(password: str, stored_hash: str) -> bool:
    try:
        salt, expected_hex = stored_hash.split("$", 1)
    except ValueError:
        return False
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt.encode("utf-8"),
100_000)
    return hmac.compare_digest(dk.hex(), expected_hex)


def create_access_token(username: str, admin_id: int) -> str:
    payload = {
        "sub": username,
        "admin_id": admin_id,
        "role": "admin",
        "exp": datetime.now(timezone.utc) + timedelta(hours=12),
    }
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)


def get_current_admin(
    credentials: HTTPAuthorizationCredentials | None = Depends(security),
) -> dict:
    if not credentials or not credentials.credentials:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token admin tidak ditemukan. Silakan login terlebih dahulu.",
        )
    try:
        payload = jwt.decode(credentials.credentials, JWT_SECRET,
algorithms=[JWT_ALGORITHM])
        if payload.get("role") != "admin":
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Akses
khusus admin.")
        return payload
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Sesi login
telah berakhir.")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Token
tidak valid.")
```

## Langkah 7: Membuat Modul Database SQLite (app/database.py)

File `app/database.py` mendefinisikan skema tabel `roles`, `admins`, `documents` (dilengkapi kolom `is_active`), `document_roles`, `document_chunks`, dan `groq_usage_logs`), sekaligus membuat akun admin dan role default `mahasiswa` secara otomatis.

**app/database.py**

```
import os
import sqlite3
from datetime import datetime, timezone
from dotenv import load_dotenv

load_dotenv()

DB_PATH = os.getenv("DB_PATH", "modul2ai.db")


def get_connection() -> sqlite3.Connection:
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON;")
    return conn


def init_db() -> None:
    from app.auth import hash_password

    conn = get_connection()
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS roles (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT UNIQUE NOT NULL,
            description TEXT DEFAULT '',
            created_at TEXT NOT NULL
        );
    """)

    cur.execute("""
        CREATE TABLE IF NOT EXISTS admins (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            created_at TEXT NOT NULL
        );
    """)

    cur.execute("""
        CREATE TABLE IF NOT EXISTS documents (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            filename TEXT NOT NULL,
            file_type TEXT NOT NULL,
            total_chunks INTEGER DEFAULT 0,
            is_active INTEGER NOT NULL DEFAULT 1,
            uploaded_at TEXT NOT NULL
        );
    """)

    cur.execute("PRAGMA table_info(documents)")
    doc_cols = {row["name"] for row in cur.fetchall()}
    if "is_active" not in doc_cols:
        cur.execute("ALTER TABLE documents ADD COLUMN is_active INTEGER NOT NULL DEFAULT
1")

    cur.execute("""
        CREATE TABLE IF NOT EXISTS document_roles (
```

```
            document_id INTEGER NOT NULL,
            role_id INTEGER NOT NULL,
            PRIMARY KEY (document_id, role_id),
            FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE CASCADE,
            FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
        );
    """)

    cur.execute("""
        CREATE TABLE IF NOT EXISTS document_chunks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            document_id INTEGER NOT NULL,
            chunk_index INTEGER NOT NULL,
            text TEXT NOT NULL,
            embedding_json TEXT NOT NULL,
            FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE CASCADE
        );
    """)

    cur.execute("""
        CREATE TABLE IF NOT EXISTS groq_usage_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            model TEXT NOT NULL,
            prompt_tokens INTEGER DEFAULT 0,
            completion_tokens INTEGER DEFAULT 0,
            total_tokens INTEGER DEFAULT 0,
            limit_requests INTEGER,
            remaining_requests INTEGER,
            reset_requests TEXT DEFAULT '',
            limit_tokens INTEGER,
            remaining_tokens INTEGER,
            reset_tokens TEXT DEFAULT '',
            status TEXT DEFAULT 'success',
            created_at TEXT NOT NULL
        );
    """)

    now = datetime.now(timezone.utc).isoformat()

    default_roles = [
        ("mahasiswa", "Akses informasi akademik & layanan SSC untuk mahasiswa"),
    ]
    for role_name, desc in default_roles:
        cur.execute(
            "INSERT OR IGNORE INTO roles (name, description, created_at) VALUES (?, ?,
?)",
            (role_name, desc, now),
        )

    admin_username = os.getenv("ADMIN_USERNAME", "admin")
    admin_password = os.getenv("ADMIN_PASSWORD", "admin123")
    cur.execute("SELECT id FROM admins WHERE username = ?", (admin_username,))
    if not cur.fetchone():
        cur.execute(
            "INSERT INTO admins (username, password_hash, created_at) VALUES (?, ?, ?)",
            (admin_username, hash_password(admin_password), now),
        )

    conn.commit()
    conn.close()
```

## Langkah 8: Membuat Modul Ekstraksi Dokumen (app/extractor.py)

File `app/extractor.py` bertugas membaca file **PDF, DOCX, XLSX, CSV, TXT, dan MD**. Pada file PDF dilakukan normalisasi karakter Unicode (`NFKC`), pembersihan ligatur font, dan perapian rumus pecahan. Sedangkan tabel pada DOCX, XLSX, dan CSV dikonversi otomatis menjadi format tabel Markdown agar mudah dipahami oleh LLM maupun ditampilkan di antarmuka.

**app/extractor.py**

```
import csv
import io
import re
import unicodedata
import zipfile
import xml.etree.ElementTree as ET
from pypdf import PdfReader
from docx import Document as DocxDocument

LIGATURE_MAP = {
    "": "ft",
    "": "ti",
    "ff": "ff",
    "fi": "fi",
    "fl": "fl",
    "ffi": "ffi",
    "ffl": "ffl",
}


def clean_extracted_text(text: str) -> str:
    text = unicodedata.normalize("NFKC", text)
    for bad, rep in LIGATURE_MAP.items():
        text = text.replace(bad, rep)
    # Hapus sisa karakter Private Use Area (PUA) yang tidak terbaca
    text = re.sub(r"[-]", "", text)
    # Rapikan pecahan rumus IKK yang terpecah antarbaris saat ekstraksi PDF
    text = re.sub(r"IKK\s*=\s*\n\s*Ni\s*\n\s*Nm\s*[×x]\s*4", "IKK = (Ni / Nm) × 4",
text)
    text = re.sub(r"Ni\s*\n\s*IKK\s*=\s*[×x]\s*4\s*\n\s*Nm", "IKK = (Ni / Nm) × 4",
text)
    return text.strip()


def _rows_to_markdown_table(rows: list[list[str]]) -> str:
    cleaned_rows = []
    for row in rows:
        cells = [re.sub(r"\s+", " ", str(c or "")).strip().replace("|", "\\|") for c in
row]
        if any(cells):
            cleaned_rows.append(cells)
    if not cleaned_rows:
        return ""

    max_cols = max(len(r) for r in cleaned_rows)
    padded = [r + [""] * (max_cols - len(r)) for r in cleaned_rows]
    header = "| " + " | ".join(padded[0]) + " |"
    sep = "| " + " | ".join(["---"] * max_cols) + " |"
    body = ["| " + " | ".join(r) + " |" for r in padded[1:]]
    return "\n".join([header, sep, *body])


def _col_ref_to_index(ref: str) -> int:
```

```
    letters = "".join(ch for ch in ref if ch.isalpha()).upper()
    idx = 0
    for ch in letters:
        idx = idx * 26 + (ord(ch) - ord("A") + 1)
    return max(0, idx - 1)


def _extract_xlsx_bytes(file_bytes: bytes) -> str:
    ns_main = "http://schemas.openxmlformats.org/spreadsheetml/2006/main"
    ns_rel = "http://schemas.openxmlformats.org/officeDocument/2006/relationships"

    with zipfile.ZipFile(io.BytesIO(file_bytes)) as zf:
        shared_strings: list[str] = []
        if "xl/sharedStrings.xml" in zf.namelist():
            root = ET.fromstring(zf.read("xl/sharedStrings.xml"))
            for si in root.findall(f"{{{ns_main}}}si"):
                text_parts = [t.text or "" for t in si.iter(f"{{{ns_main}}}t")]
                shared_strings.append("".join(text_parts))

        sheet_entries: list[tuple[str, str]] = []
        if "xl/workbook.xml" in zf.namelist() and "xl/_rels/workbook.xml.rels" in
zf.namelist():
            wb_root = ET.fromstring(zf.read("xl/workbook.xml"))
            rels_root = ET.fromstring(zf.read("xl/_rels/workbook.xml.rels"))
            rel_map = {
                rel.attrib.get("Id", ""): rel.attrib.get("Target", "").lstrip("/")
                for rel in rels_root
            }
            for sheet in wb_root.findall(f".//{{{ns_main}}}sheet"):
                name = sheet.attrib.get("name", "Sheet")
                rid = sheet.attrib.get(f"{{{ns_rel}}}id", "")
                target = rel_map.get(rid, "")
                if target and not target.startswith("xl/"):
                    target = f"xl/{target}"
                if target in zf.namelist():
                    sheet_entries.append((name, target))

        if not sheet_entries:
            for name in sorted(zf.namelist()):
                if name.startswith("xl/worksheets/sheet") and name.endswith(".xml"):
                    sheet_entries.append((name.rsplit("/", 1)[-1].replace(".xml", ""),
name))

        sections: list[str] = []
        for sheet_name, sheet_path in sheet_entries:
            s_root = ET.fromstring(zf.read(sheet_path))
            rows_data: list[list[str]] = []
            for row_el in s_root.findall(f".//{{{ns_main}}}row"):
                row_vals: dict[int, str] = {}
                for cell in row_el.findall(f"{{{ns_main}}}c"):
                    ref = cell.attrib.get("r", "")
                    col_idx = _col_ref_to_index(ref) if ref else len(row_vals)
                    c_type = cell.attrib.get("t", "")
                    val = ""
                    if c_type == "inlineStr":
                        val = "".join(t.text or "" for t in
cell.iter(f"{{{ns_main}}}t"))
                    else:
                        v_el = cell.find(f"{{{ns_main}}}v")
                        if v_el is not None and v_el.text is not None:
                            raw_val = v_el.text
                            if c_type == "s":
```

```
                                try:
                                    val = shared_strings[int(raw_val)]
                                except (ValueError, IndexError):
                                    val = raw_val
                            elif c_type == "b":
                                val = "TRUE" if raw_val == "1" else "FALSE"
                            else:
                                val = raw_val
                        row_vals[col_idx] = val
                    if row_vals:
                        max_c = max(row_vals.keys())
                        rows_data.append([row_vals.get(i, "") for i in range(max_c + 1)])

            md_table = _rows_to_markdown_table(rows_data)
            if md_table:
                if len(sheet_entries) > 1:
                    sections.append(f"### {sheet_name}\n\n{md_table}")
                else:
                    sections.append(md_table)

        return "\n\n".join(sections).strip()


def extract_text_from_bytes(file_bytes: bytes, filename: str) -> str:
    ext = filename.lower().rsplit(".", 1)[-1] if "." in filename else ""

    if ext == "pdf":
        reader = PdfReader(io.BytesIO(file_bytes))
        pages = [page.extract_text() or "" for page in reader.pages]
        return clean_extracted_text("\n".join(pages))

    if ext == "docx":
        doc = DocxDocument(io.BytesIO(file_bytes))
        parts = [p.text for p in doc.paragraphs if p.text.strip()]
        for table in doc.tables:
            table_rows = [[cell.text.strip() for cell in row.cells] for row in
table.rows]
            md_table = _rows_to_markdown_table(table_rows)
            if md_table:
                parts.append(md_table)
        return "\n\n".join(parts).strip()

    if ext == "xlsx":
        return _extract_xlsx_bytes(file_bytes)

    if ext == "csv":
        decoded = file_bytes.decode("utf-8-sig", errors="ignore").strip()
        if not decoded:
            return ""
        rows = list(csv.reader(io.StringIO(decoded)))
        md_table = _rows_to_markdown_table(rows)
        return md_table or decoded

    if ext in {"txt", "md"}:
        return file_bytes.decode("utf-8-sig", errors="ignore").strip()

    raise ValueError(f"Format file '.{ext}' belum didukung. Gunakan PDF, DOCX, XLSX,
CSV, atau TXT.")
```

## Langkah 9: Membuat Mesin RAG & Tracking Kuota Groq (app/rag.py)

File `app/rag.py` mengimplementasikan: (1) *table-aware chunking* agar tabel Markdown dan link panjang tidak terpotong sembarangan; (2) *feature hashing embedding* 384 dimensi yang ringan tanpa perlu mengunduh model embedding eksternal; (3) pencarian `search_chunks_by_role()` yang memfilter dokumen berdasarkan `role` dan status `is_active = 1`; serta (4) pemanggilan Groq LLM dengan pencatatan header *rate-limit* ke tabel `groq_usage_logs`.

**app/rag.py**

```
import json
import math
import os
import re
from datetime import datetime, timezone
from typing import Any

from dotenv import load_dotenv
from groq import Groq

from app.database import get_connection

load_dotenv()

VECTOR_SIZE = 384


def normalize_text(text: str) -> str:
    text = text.lower()
    text = re.sub(r"[^\w\s]", " ", text, flags=re.UNICODE)
    text = re.sub(r"\s+", " ", text).strip()
    return text


def _hash_token(token: str) -> int:
    h = 2166136261
    for ch in token:
        h ^= ord(ch)
        h = (h * 16777619) & 0xFFFFFFFF
    return h


def generate_embedding(text: str) -> list[float]:
    normalized = normalize_text(text)
    vector = [0.0] * VECTOR_SIZE
    if not normalized:
        return vector

    words = normalized.split()
    for word in words:
        vector[_hash_token(f"w:{word}") % VECTOR_SIZE] += 2.0
        padded = f" {word} "
        for n in (3, 4, 5):
            if len(padded) >= n:
                for i in range(len(padded) - n + 1):
                    ngram = padded[i : i + n]
                    vector[_hash_token(f"ng:{ngram}") % VECTOR_SIZE] += 0.5

    for i in range(len(words) - 1):
        bigram = f"{words[i]}_{words[i + 1]}"
        vector[_hash_token(f"bg:{bigram}") % VECTOR_SIZE] += 1.5
```

```
    norm = math.sqrt(sum(v * v for v in vector))
    if norm == 0:
        return vector
    return [v / norm for v in vector]


def cosine_similarity(a: list[float], b: list[float]) -> float:
    if not a or not b or len(a) != len(b):
        return 0.0
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot / (norm_a * norm_b)


STOPWORDS_ID = {
    "apa", "itu", "ini", "yang", "dan", "atau", "di", "ke", "dari", "untuk",
    "dengan", "adalah", "ialah", "merupakan", "ya", "kah", "dong", "sih", "nih",
    "bagaimana", "gimana", "kenapa", "mengapa", "bedanya", "perbedaan", "sama",
    "antara", "cara", "berapa", "kapan", "siapa", "dimana", "mana", "saja",
    "sebutkan", "jelaskan", "tolong", "coba", "bisa", "mau", "tanya", "tentang",
    "soal", "mengenai", "terkait", "kalau", "kalo", "jika", "bila", "apakah",
}


def lexical_bonus(question: str, title: str, chunk_text: str) -> float:
    raw_words = normalize_text(question).split()
    q_words = [w for w in raw_words if len(w) >= 2 and w not in STOPWORDS_ID]
    if not q_words:
        q_words = [w for w in raw_words if len(w) >= 2]
    if not q_words:
        return 0.0

    norm_title = normalize_text(title)
    norm_chunk = normalize_text(chunk_text)
    title_tokens = set(norm_title.split())
    chunk_tokens = set(norm_chunk.split())

    title_hits = sum(1 for w in q_words if w in title_tokens)
    text_hits = sum(1 for w in q_words if w in chunk_tokens)

    phrase = " ".join(q_words)
    phrase_bonus = 0.25 if len(q_words) >= 2 and phrase in norm_chunk else 0.0

    return (
        min(text_hits / len(q_words), 1.0) * 0.25
        + min(title_hits / len(q_words), 1.0) * 0.20
        + phrase_bonus
    )


def _split_long_block(block: str, chunk_size: int = 900, overlap: int = 150) ->
list[str]:
    if len(block) <= chunk_size:
        return [block]

    chunks: list[str] = []
    start = 0
    length = len(block)
```

```
    while start < length:
        end = min(start + chunk_size, length)
        if end < length:
            min_split = start + int(chunk_size * 0.5)
            split_pos = block.rfind("\n\n", min_split, end)
            if split_pos == -1:
                split_pos = block.rfind("\n", min_split, end)
            if split_pos == -1:
                split_pos = block.rfind(" ", min_split, end)

            if split_pos != -1:
                end = split_pos
            else:
                while end < length and not block[end].isspace():
                    end += 1

        piece = block[start:end].strip()
        if piece:
            chunks.append(piece)

        if end >= length:
            break

        next_start = max(start + 1, end - overlap)
        if next_start > 0 and not block[next_start - 1].isspace():
            while next_start < length and not block[next_start].isspace():
                next_start += 1
            while next_start < length and block[next_start].isspace():
                next_start += 1

        start = next_start if next_start < length else end

    return chunks


def chunk_text(text: str, chunk_size: int = 900, overlap: int = 150) -> list[str]:
    clean = re.sub(r"\r\n?", "\n", text).strip()
    clean = re.sub(r"\n{3,}", "\n\n", clean)
    if not clean:
        return []

    # Kelompokkan teks menjadi blok-blok logis agar tabel Markdown tidak pernah
terpotong
    blocks: list[tuple[str, bool]] = []  # (content, is_table)
    lines = clean.split("\n")
    buf: list[str] = []
    in_table = False

    def flush_buf(is_tbl: bool) -> None:
        if buf:
            content = "\n".join(buf).strip()
            if content:
                blocks.append((content, is_tbl))
            buf.clear()

    for line in lines:
        stripped = line.strip()
        is_tbl_line = stripped.startswith("|") and stripped.endswith("|") and
len(stripped) > 2
        if is_tbl_line:
            if not in_table:
                flush_buf(False)
```

```
                in_table = True
            buf.append(line)
        else:
            if in_table:
                flush_buf(True)
                in_table = False
            if stripped == "":
                flush_buf(False)
            else:
                buf.append(line)
    flush_buf(in_table)

    # Gabungkan blok-blok ke dalam chunk hingga chunk_size; blok tabel dibiarkan utuh
    chunks: list[str] = []
    current = ""

    for content, is_tbl in blocks:
        if is_tbl:
            if current and len(current) + 2 + len(content) <= max(chunk_size, 1400):
                current = f"{current}\n\n{content}"
            else:
                if current:
                    chunks.append(current)
                    current = ""
                if len(content) <= 2000:
                    chunks.append(content)
                else:
                    # Jika tabel sangat besar, pisah per baris sambil mempertahankan
header tabel
                    tbl_lines = content.split("\n")
                    header = "\n".join(tbl_lines[:2]) if len(tbl_lines) >= 2 else ""
                    sub = header
                    for row in tbl_lines[2:]:
                        if len(sub) + 1 + len(row) > chunk_size and sub != header:
                            chunks.append(sub)
                            sub = f"{header}\n{row}" if header else row
                        else:
                            sub = f"{sub}\n{row}" if sub else row
                    if sub:
                        chunks.append(sub)
            continue

        if len(content) > chunk_size:
            if current:
                chunks.append(current)
                current = ""
            chunks.extend(_split_long_block(content, chunk_size=chunk_size,
overlap=overlap))
            continue

        if not current:
            current = content
        elif len(current) + 2 + len(content) <= chunk_size:
            current = f"{current}\n\n{content}"
        else:
            chunks.append(current)
            current = content

    if current:
        chunks.append(current)

    return chunks
```

```
def search_chunks_by_role(
    question: str,
    role_name: str,
    top_k: int = 4,
    min_score: float = 0.16,
) -> list[dict[str, Any]]:
    if not question.strip():
        return []

    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT
            dc.id AS chunk_id,
            dc.document_id,
            dc.chunk_index,
            dc.text,
            dc.embedding_json,
            d.title AS document_title,
            d.filename AS document_filename
        FROM document_chunks dc
        JOIN documents d ON d.id = dc.document_id
        JOIN document_roles dr ON dr.document_id = d.id
        JOIN roles r ON r.id = dr.role_id
        WHERE LOWER(r.name) = LOWER(?) AND d.is_active = 1
        """,
        (role_name.strip(),),
    )
    rows = cur.fetchall()
    conn.close()

    if not rows:
        return []

    q_embedding = generate_embedding(question)
    scored: list[dict[str, Any]] = []

    for row in rows:
        emb = json.loads(row["embedding_json"])
        vec_score = cosine_similarity(q_embedding, emb)
        bonus = lexical_bonus(question, row["document_title"], row["text"])
        total_score = vec_score + bonus

        if total_score >= min_score:
            scored.append(
                {
                    "chunk_id": row["chunk_id"],
                    "document_id": row["document_id"],
                    "document_title": row["document_title"],
                    "document_filename": row["document_filename"],
                    "chunk_index": row["chunk_index"],
                    "text": row["text"],
                    "score": round(total_score, 4),
                }
            )

    scored.sort(key=lambda item: item["score"], reverse=True)
    return scored[:top_k]
```

```
def get_unique_sources(chunks: list[dict[str, Any]]) -> list[dict[str, Any]]:
    seen: dict[int, dict[str, Any]] = {}
    for ch in chunks:
        doc_id = ch["document_id"]
        if doc_id not in seen:
            seen[doc_id] = {
                "document_id": doc_id,
                "title": ch["document_title"],
                "filename": ch["document_filename"],
                "url": f"/uploads/{ch['document_filename']}",
                "score": ch["score"],
            }
    return list(seen.values())


GREETING_PATTERNS = {
    "hai", "halo", "hallo", "hi", "hello", "hey", "p",
    "pagi", "selamat pagi", "siang", "selamat siang",
    "sore", "selamat sore", "malam", "selamat malam",
    "assalamualaikum", "assalamu'alaikum", "permisi",
    "apa kabar", "siapa kamu", "kamu siapa",
    "terima kasih", "makasih", "thanks", "thank you", "oke", "ok", "sip",
}


def is_greeting_or_smalltalk(question: str) -> bool:
    cleaned = normalize_text(question)
    if not cleaned:
        return False
    if cleaned in GREETING_PATTERNS:
        return True
    words = cleaned.split()
    if len(words) <= 3 and any(w in {"hai", "halo", "hallo", "hi", "hello",
"assalamualaikum", "makasih", "thanks"} for w in words):
        return True
    return False


def _safe_int(val: Any) -> int | None:
    try:
        return int(val) if val is not None and str(val).strip() != "" else None
    except (ValueError, TypeError):
        return None


def _log_groq_usage(
    model: str,
    completion: Any | None = None,
    headers: Any | None = None,
    status: str = "success",
) -> None:
    try:
        usage = getattr(completion, "usage", None) if completion else None
        prompt_tokens = int(getattr(usage, "prompt_tokens", 0) or 0)
        completion_tokens = int(getattr(usage, "completion_tokens", 0) or 0)
        total_tokens = int(getattr(usage, "total_tokens", 0) or 0)

        h = {k.lower(): v for k, v in headers.items()} if headers else {}
        limit_requests = _safe_int(h.get("x-ratelimit-limit-requests"))
        remaining_requests = _safe_int(h.get("x-ratelimit-remaining-requests"))
        reset_requests = str(h.get("x-ratelimit-reset-requests", "") or "")
```

```
        limit_tokens = _safe_int(h.get("x-ratelimit-limit-tokens"))
        remaining_tokens = _safe_int(h.get("x-ratelimit-remaining-tokens"))
        reset_tokens = str(h.get("x-ratelimit-reset-tokens", "") or "")

        conn = get_connection()
        cur = conn.cursor()
        cur.execute(
            """
            INSERT INTO groq_usage_logs (
                model, prompt_tokens, completion_tokens, total_tokens,
                limit_requests, remaining_requests, reset_requests,
                limit_tokens, remaining_tokens, reset_tokens,
                status, created_at
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """,
            (
                model,
                prompt_tokens,
                completion_tokens,
                total_tokens,
                limit_requests,
                remaining_requests,
                reset_requests,
                limit_tokens,
                remaining_tokens,
                reset_tokens,
                status,
                datetime.now(timezone.utc).isoformat(),
            ),
        )
        conn.commit()
        conn.close()
    except Exception:
        pass


def generate_answer(question: str, role_name: str, relevant_chunks: list[dict[str,
Any]]) -> dict[str, Any]:
    if not relevant_chunks:
        if is_greeting_or_smalltalk(question):
            cleaned = normalize_text(question)
            if any(w in cleaned for w in ("terima kasih", "makasih", "thanks",
"thank")):
                return {
                    "answer": "Sama-sama! Jika ada pertanyaan lain seputar layanan atau
dokumen akademik, silakan tanyakan ya.",
                    "sources": [],
                }
            return {
                "answer": "Halo! Saya Asisten Dokumen Akademik. Silakan tanyakan
informasi seputar layanan akademik, jam operasional, alur menghubungi SSC, atau link
penting lainnya.",
                "sources": [],
            }
        return {
            "answer": (
                "Maaf, informasi yang Anda tanyakan belum tersedia saat ini."
            ),
            "sources": [],
        }

    sources = get_unique_sources(relevant_chunks)
```

```
    context = "\n\n---\n\n".join(
        f"[{c['document_title']}]\n{c['text']}" for c in relevant_chunks
    )

    api_key = os.getenv("GROQ_API_KEY", "").strip()
    if not api_key or api_key == "your_groq_api_key_here":
        return {
            "answer": (
                "GROQ_API_KEY belum dikonfigurasi di file .env, tetapi sistem berhasil
menemukan "
                f"{len(relevant_chunks)} potongan informasi yang relevan untuk role
'{role_name}'."
            ),
            "sources": sources,
        }

    model = os.getenv("GROQ_MODEL", "qwen/qwen3.8-27b")
    client = Groq(api_key=api_key)

    system_prompt = (
        "Kamu adalah Asisten AI Layanan Akademik (SSC) yang ramah, konsisten, dan
akurat. "
        "Jawablah pertanyaan secara langsung hanya berdasarkan informasi referensi yang
disediakan, tanpa mengarang fakta baru. "
        "ATURAN KONSISTENSI: "
        "1) Jika terdapat beberapa versi aturan (misalnya tahun 2022 dan 2025), utamakan
ketentuan terbaru (2025) sebagai acuan utama, kecuali pengguna secara eksplisit meminta
perbandingan antar versi. "
        "2) Jika istilah yang ditanyakan pengguna merujuk pada konsep yang jelas
dijelaskan dalam referensi (misalnya 'TAK Wajib' merujuk pada kategori kegiatan wajib
TAK), jelaskan langsung definisi dan rinciannya dengan tegas dan konsisten—jangan
menyatakan istilah tersebut 'tidak didefinisikan' jika penjelasannya ada di referensi. "
        "3) PENTING: Jangan pernah menyebut frasa seperti 'berdasarkan dokumen', 'dalam
dokumen', 'menurut konteks', 'dokumen yang diberikan', atau merujuk pada kata
'dokumen'/'konteks' dalam jawabanmu. "
        "Sampaikan informasi seolah-olah itu adalah pengetahuan langsung layanan SSC. "
        "Jika informasi yang ditanyakan benar-benar tidak tersedia pada referensi,
katakan dengan sopan dan singkat bahwa informasi tersebut belum tersedia saat ini. "
        "Jangan gunakan sintaks LaTeX seperti $...$, $$...$$, \\frac, atau \\times; "
        "tuliskan rumus matematika dalam teks biasa yang bersih (contoh: IKK = (Ni / Nm)
× 4)."
    )

    user_prompt = (
        f"Pertanyaan: {question}\n\n"
        f"Referensi informasi:\n{context}\n\n"
        "Jawablah pertanyaan di atas secara langsung, ringkas, dan natural dalam bahasa
Indonesia."
    )

    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ]

    used_model = model
    try:
        try:
            raw_resp = client.chat.completions.with_raw_response.create(
                model=model,
                temperature=0.0,
                max_tokens=700,
```

```
                messages=messages,
            )
            completion = raw_resp.parse()
            _log_groq_usage(used_model, completion=completion, headers=raw_resp.headers,
status="success")
        except Exception as first_exc:
            if "model_not_found" not in str(first_exc):
                raise first_exc
            available_ids = [
                m.id for m in client.models.list().data
                if not any(skip in m.id.lower() for skip in ("whisper", "guard", "tts",
"safeguard"))
            ]
            qwen_models = [mid for mid in available_ids if "qwen" in mid.lower()]
            fallback_model = qwen_models[0] if qwen_models else (available_ids[0] if
available_ids else None)
            if not fallback_model:
                raise first_exc
            used_model = fallback_model
            raw_resp = client.chat.completions.with_raw_response.create(
                model=fallback_model,
                temperature=0.0,
                max_tokens=700,
                messages=messages,
            )
            completion = raw_resp.parse()
            _log_groq_usage(used_model, completion=completion, headers=raw_resp.headers,
status="success")

        raw_answer = (completion.choices[0].message.content or "").strip()
        answer = re.sub(r"<think>.*?</think>", "", raw_answer, flags=re.DOTALL).strip()
        if not answer:
            answer = "Maaf, tidak ada respons yang dihasilkan oleh model AI."
        return {"answer": answer, "sources": sources}
    except Exception as exc:
        err_headers = getattr(getattr(exc, "response", None), "headers", None)
        _log_groq_usage(used_model, completion=None, headers=err_headers,
status="error")
        return {
            "answer": f"Terjadi kendala saat menghubungi Groq API: {exc}",
            "sources": sources,
        }
```

## Langkah 10: Membuat Backend Server Utama (app/main.py)

File `app/main.py` menyatukan seluruh modul ke dalam aplikasi FastAPI: menyajikan halaman `/` dan `/admin`, menyediakan endpoint login, CRUD *role*, upload/toggle/hapus/lihat chunks dokumen, endpoint publik `POST /api/chat`, pratinjau dokumen sumber `GET /api/public/documents/{doc_id}`, serta monitoring kuota `GET /api/admin/groq-quota`.

**app/main.py**

```
import json
import os
import re
import time
from datetime import datetime, timezone
from pathlib import Path
from typing import List
```

```
from dotenv import load_dotenv
from fastapi import Depends, FastAPI, File, Form, HTTPException, UploadFile, status
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

from app.auth import create_access_token, get_current_admin, verify_password
from app.database import get_connection, init_db
from app.extractor import extract_text_from_bytes
from app.rag import chunk_text, generate_answer, generate_embedding,
search_chunks_by_role

load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent
DATASET_DIR = BASE_DIR / "dataset"
UPLOAD_DIR = BASE_DIR / "uploads"
STATIC_DIR = BASE_DIR / "static"
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
STATIC_DIR.mkdir(parents=True, exist_ok=True)

app = FastAPI(
    title="Modul 2 AI - Custom Chatbot RAG Berbasis Role",
    version="1.0.0",
)

app.mount("/uploads", StaticFiles(directory=str(UPLOAD_DIR)), name="uploads")
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")


def seed_default_documents() -> None:
    if not DATASET_DIR.exists():
        return

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT COUNT(*) AS cnt FROM documents")
    if cur.fetchone()["cnt"] > 0:
        conn.close()
        return

    cur.execute("SELECT id FROM roles WHERE LOWER(name) = 'mahasiswa'")
    role_row = cur.fetchone()
    if not role_row:
        conn.close()
        return
    role_id = role_row["id"]

    for file_path in sorted(DATASET_DIR.iterdir()):
        if not file_path.is_file() or file_path.suffix.lower() not in {".txt", ".md",
".pdf", ".docx", ".csv", ".xlsx"}:
            continue

        file_bytes = file_path.read_bytes()
        try:
            extracted_text = extract_text_from_bytes(file_bytes, file_path.name)
        except Exception:
            continue
        if not extracted_text:
            continue

        chunks = chunk_text(extracted_text)
```

```
        if not chunks:
            continue

        safe_name = re.sub(r"[^a-zA-Z0-9._-]", "_", file_path.name)
        stored_filename = f"default_{safe_name}"
        (UPLOAD_DIR / stored_filename).write_bytes(file_bytes)

        doc_title = file_path.stem.replace("_", " ")
        ext = file_path.suffix.lstrip(".").lower() or "txt"
        now = datetime.now(timezone.utc).isoformat()

        cur.execute(
            "INSERT INTO documents (title, filename, file_type, total_chunks,
uploaded_at) VALUES (?, ?, ?, ?, ?)",
            (doc_title, stored_filename, ext, len(chunks), now),
        )
        doc_id = cur.lastrowid

        cur.execute(
            "INSERT OR IGNORE INTO document_roles (document_id, role_id) VALUES (?, ?)",
            (doc_id, role_id),
        )

        for idx, ch in enumerate(chunks):
            emb = generate_embedding(ch)
            cur.execute(
                "INSERT INTO document_chunks (document_id, chunk_index, text,
embedding_json) VALUES (?, ?, ?, ?)",
                (doc_id, idx, ch, json.dumps(emb)),
            )

    conn.commit()
    conn.close()


@app.on_event("startup")
def on_startup():
    init_db()
    seed_default_documents()


# ---------- Schemas ----------

class LoginRequest(BaseModel):
    username: str
    password: str


class RoleCreateRequest(BaseModel):
    name: str
    description: str = ""


class RoleUpdateRequest(BaseModel):
    name: str
    description: str = ""


class ChatRequest(BaseModel):
    question: str
    role: str = "mahasiswa"
```

```
# ---------- Page Routes ----------

@app.get("/", include_in_schema=False)
def serve_chat_page():
    return FileResponse(str(STATIC_DIR / "index.html"))


@app.get("/admin", include_in_schema=False)
def serve_admin_page():
    return FileResponse(str(STATIC_DIR / "admin.html"))


# ---------- Auth Routes ----------

@app.post("/api/auth/login")
def admin_login(payload: LoginRequest):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, username, password_hash FROM admins WHERE username = ?",
(payload.username.strip(),))
    admin = cur.fetchone()
    conn.close()

    if not admin or not verify_password(payload.password, admin["password_hash"]):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Username atau password admin salah.",
        )

    token = create_access_token(admin["username"], admin["id"])
    return {
        "success": True,
        "token": token,
        "username": admin["username"],
        "role": "admin",
    }


@app.get("/api/auth/me")
def get_me(admin: dict = Depends(get_current_admin)):
    return {"success": True, "admin": admin}


# ---------- Role Routes ----------

@app.get("/api/roles")
def list_roles():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT r.id, r.name, r.description, r.created_at,
               COUNT(dr.document_id) AS document_count
        FROM roles r
        LEFT JOIN document_roles dr ON dr.role_id = r.id
        GROUP BY r.id
        ORDER BY r.id ASC
        """
    )
    roles = [dict(row) for row in cur.fetchall()]
    conn.close()
```

```
    return {"success": True, "roles": roles}


@app.post("/api/roles", status_code=status.HTTP_201_CREATED)
def create_role(payload: RoleCreateRequest, _admin: dict = Depends(get_current_admin)):
    role_name = payload.name.strip().lower()
    if not role_name:
        raise HTTPException(status_code=400, detail="Nama role tidak boleh kosong.")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id FROM roles WHERE LOWER(name) = LOWER(?)", (role_name,))
    if cur.fetchone():
        conn.close()
        raise HTTPException(status_code=400, detail=f"Role '{role_name}' sudah ada.")

    now = datetime.now(timezone.utc).isoformat()
    cur.execute(
        "INSERT INTO roles (name, description, created_at) VALUES (?, ?, ?)",
        (role_name, payload.description.strip(), now),
    )
    role_id = cur.lastrowid
    conn.commit()
    conn.close()

    return {
        "success": True,
        "role": {"id": role_id, "name": role_name, "description":
payload.description.strip()},
    }


@app.put("/api/roles/{role_id}")
def update_role(role_id: int, payload: RoleUpdateRequest, _admin: dict =
Depends(get_current_admin)):
    role_name = payload.name.strip().lower()
    if not role_name:
        raise HTTPException(status_code=400, detail="Nama role tidak boleh kosong.")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id FROM roles WHERE id = ?", (role_id,))
    if not cur.fetchone():
        conn.close()
        raise HTTPException(status_code=404, detail="Role tidak ditemukan.")

    cur.execute("SELECT id FROM roles WHERE LOWER(name) = LOWER(?) AND id != ?",
(role_name, role_id))
    if cur.fetchone():
        conn.close()
        raise HTTPException(status_code=400, detail=f"Nama role '{role_name}' sudah
digunakan.")

    cur.execute(
        "UPDATE roles SET name = ?, description = ? WHERE id = ?",
        (role_name, payload.description.strip(), role_id),
    )
    conn.commit()
    conn.close()
    return {"success": True, "message": "Role berhasil diperbarui."}
```

```
@app.delete("/api/roles/{role_id}")
def delete_role(role_id: int, _admin: dict = Depends(get_current_admin)):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, name FROM roles WHERE id = ?", (role_id,))
    row = cur.fetchone()
    if not row:
        conn.close()
        raise HTTPException(status_code=404, detail="Role tidak ditemukan.")

    cur.execute("DELETE FROM roles WHERE id = ?", (role_id,))
    conn.commit()
    conn.close()
    return {"success": True, "message": f"Role '{row['name']}' berhasil dihapus."}


# ---------- Document Routes (Admin Only) ----------

@app.get("/api/documents")
def list_documents(_admin: dict = Depends(get_current_admin)):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT * FROM documents ORDER BY id DESC")
    docs = [dict(r) for r in cur.fetchall()]

    for doc in docs:
        cur.execute(
            """
            SELECT r.id, r.name
            FROM roles r
            JOIN document_roles dr ON dr.role_id = r.id
            WHERE dr.document_id = ?
            ORDER BY r.name ASC
            """,
            (doc["id"],),
        )
        doc["roles"] = [dict(r) for r in cur.fetchall()]

    conn.close()
    return {"success": True, "documents": docs}


@app.get("/api/documents/{doc_id}/chunks")
def get_document_chunks(doc_id: int, _admin: dict = Depends(get_current_admin)):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, title, filename, file_type, total_chunks FROM documents
WHERE id = ?", (doc_id,))
    doc = cur.fetchone()
    if not doc:
        conn.close()
        raise HTTPException(status_code=404, detail="Dokumen tidak ditemukan.")

    cur.execute(
        "SELECT chunk_index, text FROM document_chunks WHERE document_id = ? ORDER BY
chunk_index ASC",
        (doc_id,),
    )
    chunks = [dict(r) for r in cur.fetchall()]
    conn.close()
    return {"success": True, "document": dict(doc), "chunks": chunks}
```

```
@app.post("/api/documents/upload", status_code=status.HTTP_201_CREATED)
async def upload_document(
    file: UploadFile = File(...),
    title: str = Form(""),
    role_ids: str = Form(...),  # JSON array e.g. "[1, 2]" or comma-separated "1,2"
    _admin: dict = Depends(get_current_admin),
):
    if not file.filename:
        raise HTTPException(status_code=400, detail="File wajib dipilih.")

    try:
        parsed_ids: List[int] = (
            json.loads(role_ids) if role_ids.strip().startswith("[")
            else [int(x.strip()) for x in role_ids.split(",") if x.strip()]
        )
    except Exception:
        raise HTTPException(status_code=400, detail="Format role_ids tidak valid.")

    if not parsed_ids:
        raise HTTPException(status_code=400, detail="Pilih minimal satu role untuk
dokumen ini.")

    file_bytes = await file.read()
    try:
        extracted_text = extract_text_from_bytes(file_bytes, file.filename)
    except ValueError as err:
        raise HTTPException(status_code=400, detail=str(err))

    if not extracted_text:
        raise HTTPException(status_code=400, detail="Teks tidak dapat diekstrak dari
dokumen ini.")

    chunks = chunk_text(extracted_text)
    if not chunks:
        raise HTTPException(status_code=400, detail="Dokumen tidak menghasilkan chunk
teks.")

    safe_name = re.sub(r"[^a-zA-Z0-9._-]", "_", file.filename)
    stored_filename = f"{int(time.time())}_{safe_name}"
    (UPLOAD_DIR / stored_filename).write_bytes(file_bytes)

    doc_title = title.strip() or Path(file.filename).stem
    ext = file.filename.rsplit(".", 1)[-1].lower() if "." in file.filename else "txt"
    now = datetime.now(timezone.utc).isoformat()

    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        "INSERT INTO documents (title, filename, file_type, total_chunks, uploaded_at)
VALUES (?, ?, ?, ?, ?)",
        (doc_title, stored_filename, ext, len(chunks), now),
    )
    doc_id = cur.lastrowid

    for rid in parsed_ids:
        cur.execute("SELECT id FROM roles WHERE id = ?", (rid,))
        if cur.fetchone():
            cur.execute(
                "INSERT OR IGNORE INTO document_roles (document_id, role_id) VALUES (?,
?)",
```

```
                (doc_id, rid),
            )

    for idx, ch in enumerate(chunks):
        emb = generate_embedding(ch)
        cur.execute(
            "INSERT INTO document_chunks (document_id, chunk_index, text,
embedding_json) VALUES (?, ?, ?, ?)",
            (doc_id, idx, ch, json.dumps(emb)),
        )

    conn.commit()
    conn.close()

    return {
        "success": True,
        "message": f"Dokumen '{doc_title}' berhasil diunggah ({len(chunks)} chunks).",
        "document_id": doc_id,
    }


@app.patch("/api/documents/{doc_id}/toggle")
def toggle_document_status(doc_id: int, _admin: dict = Depends(get_current_admin)):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, title, is_active FROM documents WHERE id = ?", (doc_id,))
    doc = cur.fetchone()
    if not doc:
        conn.close()
        raise HTTPException(status_code=404, detail="Dokumen tidak ditemukan.")

    new_status = 0 if doc["is_active"] else 1
    cur.execute("UPDATE documents SET is_active = ? WHERE id = ?", (new_status, doc_id))
    conn.commit()
    conn.close()

    state_label = "diaktifkan" if new_status == 1 else "dinonaktifkan"
    return {
        "success": True,
        "is_active": bool(new_status),
        "message": f"Dokumen '{doc['title']}' berhasil {state_label}.",
    }


@app.delete("/api/documents/{doc_id}")
def delete_document(doc_id: int, _admin: dict = Depends(get_current_admin)):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, filename, title FROM documents WHERE id = ?", (doc_id,))
    doc = cur.fetchone()
    if not doc:
        conn.close()
        raise HTTPException(status_code=404, detail="Dokumen tidak ditemukan.")

    cur.execute("DELETE FROM documents WHERE id = ?", (doc_id,))
    conn.commit()
    conn.close()

    file_path = UPLOAD_DIR / doc["filename"]
    if file_path.exists():
        file_path.unlink(missing_ok=True)
```

```
    return {"success": True, "message": f"Dokumen '{doc['title']}' berhasil dihapus."}


# ---------- Chat Route (Public, Role-Filtered) ----------

@app.post("/api/chat")
def chat_with_rag(payload: ChatRequest):
    question = payload.question.strip()
    role_name = payload.role.strip().lower()

    if not question:
        raise HTTPException(status_code=400, detail="Pertanyaan tidak boleh kosong.")

    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, name FROM roles WHERE LOWER(name) = LOWER(?)", (role_name,))
    role_row = cur.fetchone()
    conn.close()

    if not role_row:
        raise HTTPException(status_code=404, detail=f"Role '{role_name}' tidak
ditemukan.")

    relevant_chunks = search_chunks_by_role(question, role_row["name"], top_k=7,
min_score=0.16)
    result = generate_answer(question, role_row["name"], relevant_chunks)

    return {
        "success": True,
        "role": role_row["name"],
        "answer": result["answer"],
        "sources": result["sources"],
    }


def _parse_duration_seconds(dur: str) -> float:
    if not dur:
        return 0.0
    total = 0.0
    for val, unit in re.findall(r"(\d+(?:\.\d+)?)(ms|h|m|s)", dur.lower()):
        num = float(val)
        if unit == "h":
            total += num * 3600
        elif unit == "m":
            total += num * 60
        elif unit == "s":
            total += num
        elif unit == "ms":
            total += num / 1000.0
    return total


def _format_remaining_duration(seconds: float) -> str:
    if seconds <= 0:
        return "Sudah reset"
    secs = int(round(seconds))
    if secs < 60:
        return f"{secs}d"
    mins, rem_s = divmod(secs, 60)
    if mins < 60:
        return f"{mins}m {rem_s}d" if rem_s else f"{mins}m"
    hours, rem_m = divmod(mins, 60)
```

```
    return f"{hours}j {rem_m}m"


@app.get("/api/admin/groq-quota")
def get_groq_quota(_admin: dict = Depends(get_current_admin)):
    api_key = os.getenv("GROQ_API_KEY", "").strip()
    configured_model = os.getenv("GROQ_MODEL", "qwen/qwen3.8-27b")
    has_api_key = bool(api_key and api_key != "your_groq_api_key_here")

    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        SELECT model, limit_requests, remaining_requests, reset_requests,
               limit_tokens, remaining_tokens, reset_tokens, created_at
        FROM groq_usage_logs
        WHERE limit_requests IS NOT NULL OR limit_tokens IS NOT NULL
        ORDER BY id DESC
        LIMIT 1
        """
    )
    latest_row = cur.fetchone()
    latest = dict(latest_row) if latest_row else None

    if latest:
        try:
            recorded_at = datetime.fromisoformat(latest["created_at"])
            elapsed = max(0.0, (datetime.now(timezone.utc) -
recorded_at).total_seconds())
        except Exception:
            elapsed = 0.0

        req_reset_sec = _parse_duration_seconds(latest.get("reset_requests", ""))
        req_rem_sec = max(0.0, req_reset_sec - elapsed)
        if latest.get("limit_requests") and req_reset_sec > 0 and req_rem_sec <= 0:
            latest["remaining_requests"] = latest["limit_requests"]
            latest["reset_requests_label"] = "Sudah reset"
        else:
            latest["reset_requests_label"] = _format_remaining_duration(req_rem_sec) if
req_reset_sec > 0 else "-"

        tok_reset_sec = _parse_duration_seconds(latest.get("reset_tokens", ""))
        tok_rem_sec = max(0.0, tok_reset_sec - elapsed)
        if latest.get("limit_tokens") and tok_reset_sec > 0 and tok_rem_sec <= 0:
            latest["remaining_tokens"] = latest["limit_tokens"]
            latest["reset_tokens_label"] = "Sudah reset"
        else:
            latest["reset_tokens_label"] = _format_remaining_duration(tok_rem_sec) if
tok_reset_sec > 0 else "-"

    today_prefix = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    cur.execute(
        """
        SELECT COUNT(*) AS requests_today,
               COALESCE(SUM(prompt_tokens), 0) AS prompt_tokens_today,
               COALESCE(SUM(completion_tokens), 0) AS completion_tokens_today,
               COALESCE(SUM(total_tokens), 0) AS total_tokens_today
        FROM groq_usage_logs
        WHERE created_at LIKE ?
        """,
        (f"{today_prefix}%",),
```

```
    )
    today_row = dict(cur.fetchone())

    cur.execute(
        """
        SELECT COUNT(*) AS total_requests,
               COALESCE(SUM(total_tokens), 0) AS total_tokens
        FROM groq_usage_logs
        """
    )
    total_row = dict(cur.fetchone())

    cur.execute(
        """
        SELECT id, model, prompt_tokens, completion_tokens, total_tokens, status,
created_at
        FROM groq_usage_logs
        ORDER BY id DESC
        LIMIT 10
        """
    )
    recent_logs = [dict(r) for r in cur.fetchall()]
    conn.close()

    return {
        "success": True,
        "has_api_key": has_api_key,
        "configured_model": configured_model,
        "latest": latest,
        "today": today_row,
        "total": total_row,
        "recent_logs": recent_logs,
    }


@app.get("/api/public/documents/{doc_id}")
def get_public_document(doc_id: int):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT d.id, d.title, d.filename, d.file_type, d.total_chunks
        FROM documents d
        JOIN document_roles dr ON dr.document_id = d.id
        JOIN roles r ON r.id = dr.role_id
        WHERE d.id = ? AND LOWER(r.name) = 'mahasiswa' AND d.is_active = 1
        """,
        (doc_id,),
    )
    doc = cur.fetchone()
    if not doc:
        conn.close()
        raise HTTPException(status_code=404, detail="Dokumen tidak ditemukan.")

    file_path = UPLOAD_DIR / doc["filename"]
    full_text = ""
    if file_path.exists():
        try:
            full_text = extract_text_from_bytes(file_path.read_bytes(), doc["filename"])
        except Exception:
            full_text = ""
```

```
    if not full_text:
        cur.execute(
            "SELECT text FROM document_chunks WHERE document_id = ? ORDER BY chunk_index
ASC",
            (doc_id,),
        )
        full_text = "\n\n".join(r["text"] for r in cur.fetchall())

    conn.close()

    return {
        "success": True,
        "document": dict(doc),
        "url": f"/uploads/{doc['filename']}",
        "content": full_text,
    }
```

## Langkah 11: Membuat Antarmuka Chatbot Mahasiswa (static/index.html)

File `static/index.html` menyediakan halaman utama chatbot bagi mahasiswa tanpa perlu login. Fitur utamanya meliputi kartu sambutan dengan 4 pertanyaan rekomendasi yang langsung terkirim saat diklik, penyimpanan riwayat chat di `localStorage`, tombol **+ Chat Baru**, renderer Markdown XSS-safe (mendukung tabel responsif, link klik, daftar bernomor, dan rumus matematika bersih), serta popup pratinjau dokumen saat sumber referensi diklik.

**static/index.html**

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Chatbot RAG — Asisten Dokumen Akademik</title>
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet" />
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          fontFamily: { sans: ['Inter', 'system-ui', 'sans-serif'] }
        }
      }
    }
  </script>
  <style>
    .bg-grid {
      background-image: radial-gradient(rgba(99, 102, 241, 0.08) 1px, transparent 1px);
      background-size: 24px 24px;
    }
  </style>
</head>
<body class="bg-[#FAFAFB] bg-grid text-zinc-900 font-sans antialiased min-h-screen flex
flex-col">

  <!-- Top Navbar -->
```

```
  <header class="bg-white/85 backdrop-blur-md border-b border-zinc-200/75 sticky top-0
z-20">
    <div class="max-w-4xl mx-auto px-4 sm:px-6 h-14 flex items-center justify-between
gap-3">
      <div class="flex items-center gap-2.5">
        <div class="w-8 h-8 rounded-xl bg-gradient-to-br from-indigo-600 to-violet-600
text-white flex items-center justify-center shadow-sm shadow-indigo-500/25">
          <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M8 10h.01M12
10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5
5v-5z" />
          </svg>
        </div>
        <div class="leading-tight">
          <span class="text-sm font-semibold tracking-tight text-zinc-900 block">Chatbot
RAG</span>
          <span class="text-[11px] text-zinc-500 block">Asisten Dokumen Akademik</span>
        </div>
      </div>

      <div class="flex items-center gap-2">
        <button id="clearChatBtn" type="button" onclick="clearPublicChatHistory()"
class="hidden text-xs font-medium text-indigo-600 bg-indigo-50 hover:bg-indigo-100 px-3
py-1.5 rounded-xl transition cursor-pointer">
          + Chat Baru
        </button>
        <a href="/admin" class="inline-flex items-center gap-1.5 text-xs font-medium px-
3 py-1.5 rounded-xl bg-zinc-900 hover:bg-zinc-800 text-white shadow-xs transition">
          <svg class="w-3.5 h-3.5 text-zinc-300" fill="none" stroke="currentColor"
stroke-width="2" viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M12 15v2m-6 4h12a2 2
0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z" />
          </svg>
          Login Admin
        </a>
      </div>
    </div>
  </header>

  <!-- Main Chat Area -->
  <main class="flex-1 max-w-4xl w-full mx-auto px-4 sm:px-6 py-6 flex flex-col">
    <div id="chatBox" class="flex-1 space-y-5 overflow-y-auto pb-6">
      <!-- Hero Welcome Card -->
      <div id="welcomeState" class="bg-white border border-zinc-200/80 rounded-2xl p-6
shadow-[0_1px_3px_rgba(0,0,0,0.03)]">
        <div class="inline-flex items-center gap-1.5 text-[11px] font-medium text-
indigo-700 bg-indigo-50 border border-indigo-100 px-2.5 py-0.5 rounded-full mb-3">
          <span class="w-1.5 h-1.5 rounded-full bg-indigo-500 animate-pulse"></span>
          Siap Membantu
        </div>
        <h1 class="text-lg font-semibold tracking-tight text-zinc-900">Tanyakan Layanan
&amp;amp; Dokumen Akademik</h1>
        <p class="text-sm text-zinc-500 mt-1 leading-relaxed max-w-2xl">
          Asisten ini menjawab pertanyaan berdasarkan dokumen akademik yang tersedia dan
selalu menyertakan sumber referensi di setiap jawaban.
        </p>

        <div class="mt-5 grid grid-cols-1 sm:grid-cols-2 gap-2.5">
          <button type="button" onclick="usePrompt('Apa saja ketentuan dan nilai minimal
TAK mahasiswa menurut SK TAK terbaru?')"
```

```
            class="text-left p-3 rounded-xl border border-zinc-200/80 bg-zinc-50/70
hover:bg-indigo-50/50 hover:border-indigo-200 transition group">
            <div class="text-xs font-medium text-zinc-800 group-hover:text-indigo-
700">Pedoman &amp;amp; Ketentuan TAK</div>
            <div class="text-[11px] text-zinc-400 mt-0.5">Nilai minimal &amp;amp; aturan SK
TAK 2025</div>
          </button>
          <button type="button" onclick="usePrompt('Apa perubahan aturan TAK pada SK
2025 dibandingkan SK TAK 2022?')"
            class="text-left p-3 rounded-xl border border-zinc-200/80 bg-zinc-50/70
hover:bg-indigo-50/50 hover:border-indigo-200 transition group">
            <div class="text-xs font-medium text-zinc-800 group-hover:text-indigo-
700">Perubahan SK TAK 2022 &amp;amp; 2025</div>
            <div class="text-[11px] text-zinc-400 mt-0.5">Bandingkan pedoman TAK lama
&amp;amp; baru</div>
          </button>
          <button type="button" onclick="usePrompt('Kapan jam operasional layanan SSC
Telkom University?')"
            class="text-left p-3 rounded-xl border border-zinc-200/80 bg-zinc-50/70
hover:bg-indigo-50/50 hover:border-indigo-200 transition group">
            <div class="text-xs font-medium text-zinc-800 group-hover:text-indigo-
700">Jam Operasional SSC</div>
            <div class="text-[11px] text-zinc-400 mt-0.5">Hari &amp;amp; jam buka layanan
mahasiswa</div>
          </button>
          <button type="button" onclick="usePrompt('Bagaimana alur menghubungi layanan
SSC?')"
            class="text-left p-3 rounded-xl border border-zinc-200/80 bg-zinc-50/70
hover:bg-indigo-50/50 hover:border-indigo-200 transition group">
            <div class="text-xs font-medium text-zinc-800 group-hover:text-indigo-
700">Alur Menghubungi SSC</div>
            <div class="text-[11px] text-zinc-400 mt-0.5">Kontak WhatsApp, email, &amp;amp;
layanan</div>
          </button>
        </div>
      </div>
    </div>

    <!-- Floating Composer -->
    <div class="sticky bottom-4 pt-2">
      <form id="chatForm" class="bg-white border border-zinc-200/90 rounded-2xl p-1.5
flex items-center gap-2 shadow-lg shadow-zinc-200/40 focus-within:ring-2 focus-
within:ring-indigo-500/20 focus-within:border-indigo-400 transition">
        <input
          id="questionInput"
          type="text"
          placeholder="Tulis pertanyaan Anda di sini..."
          autocomplete="off"
          class="flex-1 px-3.5 py-2 text-sm text-zinc-900 placeholder:text-zinc-400
focus:outline-none bg-transparent"
          required
        />
        <button
          id="sendBtn"
          type="submit"
          class="inline-flex items-center gap-1.5 bg-indigo-600 hover:bg-indigo-700
disabled:opacity-40 text-white text-xs font-medium px-4 py-2.5 rounded-xl shadow-xs
transition cursor-pointer"
        >
          <span>Kirim</span>
          <svg class="w-3.5 h-3.5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
```

```
            <path stroke-linecap="round" stroke-linejoin="round" d="M22 2L11 13M22 2l-7
20-4-9-9-4 20-7z" />
          </svg>
        </button>
      </form>
    </div>
  </main>

  <!-- Source Preview Modal Popup -->
  <div id="sourceModal" class="hidden fixed inset-0 z-50 bg-zinc-900/50 backdrop-blur-xs
flex items-center justify-center p-4">
    <div class="bg-white border border-zinc-200 rounded-2xl shadow-2xl w-full max-w-2xl
max-h-[82vh] flex flex-col overflow-hidden">
      <div class="px-5 py-4 border-b border-zinc-100 flex items-center justify-between
gap-3">
        <div class="min-w-0 flex-1">
          <h3 id="sourceModalTitle" class="text-sm font-semibold text-zinc-900
truncate">Sumber Dokumen</h3>
          <p id="sourceModalMeta" class="text-xs text-zinc-500 mt-0.5"></p>
        </div>
        <button type="button" onclick="closeSourceModal()" class="text-zinc-400
hover:text-zinc-700 p-1.5 rounded-lg hover:bg-zinc-100 transition cursor-pointer">
          <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M6 18L18 6M6 6l12
12" />
          </svg>
        </button>
      </div>
      <div id="sourceModalBody" class="p-5 overflow-y-auto overflow-x-hidden space-y-3
bg-zinc-50/50 flex-1 text-xs leading-relaxed text-zinc-700 break-words [overflow-
wrap:anywhere]"></div>
      <div class="px-5 py-3 border-t border-zinc-100 bg-white flex items-center justify-
between gap-2">
        <a id="sourceModalDownload" href="#" download class="hidden text-xs font-medium
text-indigo-600 hover:text-indigo-700 px-3 py-1.5 rounded-lg hover:bg-indigo-50
transition">
          Unduh File Asli
        </a>
        <button type="button" onclick="closeSourceModal()" class="ml-auto text-xs font-
medium text-zinc-700 bg-zinc-100 hover:bg-zinc-200 px-4 py-2 rounded-xl transition
cursor-pointer">
          Tutup
        </button>
      </div>
    </div>
  </div>

  <script>
    const PUBLIC_CHAT_KEY = "public_chat_history";
    const chatBox = document.getElementById("chatBox");
    const chatForm = document.getElementById("chatForm");
    const questionInput = document.getElementById("questionInput");
    const sendBtn = document.getElementById("sendBtn");
    const clearChatBtn = document.getElementById("clearChatBtn");
    const welcomeHtml = document.getElementById("welcomeState").outerHTML;

    function getPublicHistory() {
      try {
        return JSON.parse(localStorage.getItem(PUBLIC_CHAT_KEY) || "[]");
      } catch {
        return [];
```

```
    }
  }

  function savePublicHistory(items) {
    localStorage.setItem(PUBLIC_CHAT_KEY, JSON.stringify(items));
    if (items.length > 0) clearChatBtn.classList.remove("hidden");
    else clearChatBtn.classList.add("hidden");
  }

  window.clearPublicChatHistory = function () {
    localStorage.removeItem(PUBLIC_CHAT_KEY);
    clearChatBtn.classList.add("hidden");
    chatBox.innerHTML = welcomeHtml;
  };

  function escapeHtml(str) {
    return String(str)
      .replace(/&/g, "&amp;")
      .replace(/</g, "&lt;")
      .replace(/>/g, "&gt;")
      .replace(/"/g, "&quot;");
  }

  function formatMarkdown(text) {
    const escaped = escapeHtml(text || "");
    const lines = escaped.split(/\r?\n/);
    let html = "";
    let inUl = false;
    let inOl = false;

    function cleanLatex(expr) {
      return expr
        .replace(/\\left\s*([(\[{|])/g, "$1")
        .replace(/\\right\s*([)\]}|])/g, "$1")
        .replace(/\\frac\s*\{([^{}]+)\}\s*\{([^{}]+)\}/g, "$1 / $2")
        .replace(/\\times/g, "×")
        .replace(/\\div/g, "÷")
        .replace(/\\cdot/g, "·")
        .replace(/\\pm/g, "±")
        .replace(/\\leq/g, "≤")
        .replace(/\\geq/g, "≥")
        .replace(/\\neq/g, "≠")
        .replace(/\\approx/g, "≈")
        .replace(/\\%/g, "%")
        .replace(/_\{([^{}]+)\}/g, "$1")
        .replace(/\^\{([^{}]+)\}/g, "^$1")
        .replace(/\(\s+/g, "(")
        .replace(/\s+\)/g, ")")
        .replace(/\s{2,}/g, " ")
        .trim();
    }

    function inlineFormat(s) {
      const links = [];
      const stashLink = (href, label, external = true) => {
        const idx = links.length;
        const attrs = external ? ' target="_blank" rel="noopener noreferrer"' : "";
        links.push(`<a href="${href}"${attrs} class="text-indigo-600 hover:underline
break-all">${label}</a>`);
        return `\u0000LINK${idx}\u0000`;
      };
```

```
        let out = s
          .replace(/\[([^\]]+)\]\(((?:https?:\/\/|mailto:)[^\s)]+)\)/g, (_, label, url)
=>
            stashLink(url, label, url.startsWith("http"))
          )
          .replace(/(^|[\s(>])(https?:\/\/[^\s<)]+)/g, (_, pre, url) => {
            const cleanUrl = url.replace(/[.,;:!?]+$/, "");
            const trail = url.slice(cleanUrl.length);
            return pre + stashLink(cleanUrl, cleanUrl, true) + trail;
          })
          .replace(/(^|[\s(>])([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})/g, (_,
pre, email) =>
            pre + stashLink(`mailto:${email}`, email, false)
          )
          .replace(/(^|[\s(>])((?:linktr\.ee|(?:[a-zA-Z0-9-
]+\.)+(?:ac\.id|id|com|org|edu|net))(?:\/[^\s<)]*)?)/g, (_, pre, domainUrl) => {
            const cleanUrl = domainUrl.replace(/[.,;:!?]+$/, "");
            const trail = domainUrl.slice(cleanUrl.length);
            return pre + stashLink(`https://${cleanUrl}`, cleanUrl, true) + trail;
          })
          .replace(/\*\*([^*]+)\*\*/g, '<strong class="font-semibold text-zinc-
900">$1</strong>')
          .replace(/(^|[^\*])\*([^*\n]+)\*/g, '$1$2')
          .replace(/`([^`]+)`/g, '<code class="bg-zinc-100 text-zinc-800 px-1 py-0.5
rounded text-xs">$1</code>')
          .replace(/\$\$([^$]+)\$\$/g, (_, math) => `<span class="font-mono text-xs
font-medium text-zinc-900 bg-zinc-100 px-1.5 py-0.5
rounded">${cleanLatex(math)}</span>`)
          .replace(/\$([^$\n]+)\$/g, (_, math) => `<span class="font-mono text-xs font-
medium text-zinc-900 bg-zinc-100 px-1 py-0.5 rounded">${cleanLatex(math)}</span>`)
          .replace(/\\\((.+?)\\\)/g, (_, math) => `<span class="font-mono text-xs font-
medium text-zinc-900 bg-zinc-100 px-1 py-0.5 rounded">${cleanLatex(math)}</span>`);

        return out.replace(/\u0000LINK(\d+)\u0000/g, (_, i) => links[Number(i)]);
      }

      function closeLists() {
        if (inUl) { html += "</ul>"; inUl = false; }
        if (inOl) { html += "</ol>"; inOl = false; }
      }

      function isTableRow(line) {
        return line.startsWith("|") && line.endsWith("|") && line.length > 2;
      }

      function isTableSeparator(line) {
        return /^\|[\s:\-\|]+\|$/.test(line);
      }

      function parseTableCells(line) {
        return line.slice(1, -1).split("|").map((c) => c.trim());
      }

      for (let i = 0; i < lines.length; i++) {
        const line = lines[i].trim();
        if (!line) {
          closeLists();
          continue;
        }

        if (/^(-{3,}|\*{3,})$/.test(line)) {
          closeLists();
```

```
          html += '<hr class="my-3 border-zinc-200/80" />';
          continue;
        }

        const blockMathMatch = line.match(/^(?:\$\$(.+)\$\$|\\\[(.+)\\\])$/);
        if (blockMathMatch) {
          closeLists();
          const expr = cleanLatex(blockMathMatch[1] || blockMathMatch[2] || "");
          html += `<div class="my-2 px-3.5 py-2.5 rounded-xl bg-zinc-50 border border-
zinc-200/80 font-mono text-xs sm:text-sm font-medium text-zinc-900 text-center overflow-
x-auto">${expr}</div>`;
          continue;
        }

        if (isTableRow(line) && i + 1 < lines.length && isTableSeparator(lines[i +
1].trim())) {
          closeLists();
          const headers = parseTableCells(line);
          i += 1; // lewati baris pemisah
          let tableHtml = '<div class="w-full max-w-full overflow-x-auto my-2.5 rounded-
xl border border-zinc-200/80"><table class="w-full text-left border-collapse text-xs
break-normal [overflow-wrap:normal]">';
          tableHtml += '<thead class="bg-zinc-50 border-b border-zinc-200/80"><tr>';
          headers.forEach((h) => {
            tableHtml += `<th class="px-3 py-2 font-semibold text-zinc-800 align-bottom
leading-snug">${inlineFormat(h)}</th>`;
          });
          tableHtml += '</tr></thead><tbody class="divide-y divide-zinc-100 bg-white">';
          while (i + 1 < lines.length && isTableRow(lines[i + 1].trim())) {
            i += 1;
            const cells = parseTableCells(lines[i].trim());
            tableHtml += '<tr>';
            headers.forEach((_, colIdx) => {
              tableHtml += `<td class="px-3 py-2 text-zinc-700 align-top whitespace-
nowrap sm:whitespace-normal">${inlineFormat(cells[colIdx] || "")}</td>`;
            });
            tableHtml += '</tr>';
          }
          tableHtml += '</tbody></table></div>';
          html += tableHtml;
          continue;
        }

        const ulMatch = line.match(/^[\*\-]\s+(.*)$/);
        const olMatch = line.match(/^(\d+)\.\s+(.*)$/);
        const hMatch = line.match(/^(#{1,3})\s+(.*)$/);

        if (ulMatch) {
          if (inOl) { html += "</ol>"; inOl = false; }
          if (!inUl) { html += '<ul class="list-disc pl-5 space-y-1 my-2">'; inUl =
true; }
          html += `<li>${inlineFormat(ulMatch[1])}</li>`;
        } else if (olMatch) {
          if (inUl) { html += "</ul>"; inUl = false; }
          if (!inOl) { html += '<ol class="list-decimal pl-5 space-y-1 my-2">'; inOl =
true; }
          html += `<li value="${olMatch[1]}">${inlineFormat(olMatch[2])}</li>`;
        } else if (hMatch) {
          closeLists();
          const level = hMatch[1].length;
          const cls = level === 1 ? "text-sm font-semibold text-zinc-900 mt-3 mb-1.5" :
"text-xs font-semibold text-zinc-900 mt-2.5 mb-1";
```

```
          html += `<div class="${cls}">${inlineFormat(hMatch[2])}</div>`;
        } else {
          closeLists();
          html += `<p class="my-1.5 first:mt-0 last:mb-0">${inlineFormat(line)}</p>`;
        }
      }
      closeLists();
      return html;
    }

    window.usePrompt = function (text) {
      questionInput.value = text;
      if (typeof chatForm.requestSubmit === "function") {
        chatForm.requestSubmit();
      } else {
        chatForm.dispatchEvent(new Event("submit", { cancelable: true, bubbles: true
}));
      }
    };

    function appendUserMessage(text) {
      const wrapper = document.createElement("div");
      wrapper.className = "flex justify-end";
      wrapper.innerHTML = `
        <div class="bg-zinc-900 text-white rounded-2xl rounded-br-md px-4 py-3 max-w-xl
shadow-xs">
          <p class="text-sm leading-relaxed whitespace-pre-wrap">${escapeHtml(text)}</p>
        </div>
      `;
      chatBox.appendChild(wrapper);
      chatBox.scrollTop = chatBox.scrollHeight;
    }

    function appendLoading() {
      const el = document.createElement("div");
      el.id = "loadingBubble";
      el.className = "flex justify-start";
      el.innerHTML = `
        <div class="bg-white border border-zinc-200/80 rounded-2xl rounded-bl-md px-4
py-3 shadow-xs flex items-center gap-2.5 text-xs text-zinc-500">
          <span class="inline-flex gap-1">
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce"></span>
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce
[animation-delay:0.15s]"></span>
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce
[animation-delay:0.3s]"></span>
          </span>
          <span>Mencari dokumen &amp;amp; menyusun jawaban...</span>
        </div>
      `;
      chatBox.appendChild(el);
      chatBox.scrollTop = chatBox.scrollHeight;
    }

    function removeLoading() {
      const el = document.getElementById("loadingBubble");
      if (el) el.remove();
    }

    function appendBotMessage(answer, sources = []) {
      const wrapper = document.createElement("div");
      wrapper.className = "flex justify-start items-start gap-2.5";
```

```
      let sourcesHtml = "";
      if (sources && sources.length > 0) {
        const items = sources
          .map(
            (s) => `
            <button type="button" onclick="openSourceModal(${Number(s.document_id)},
'${escapeHtml(s.title)}')"
               class="inline-flex items-center gap-1.5 text-xs bg-zinc-50 hover:bg-
indigo-50/70 text-zinc-700 hover:text-indigo-700 px-2.5 py-1.5 rounded-lg border border-
zinc-200/80 hover:border-indigo-200 transition cursor-pointer">
              <svg class="w-3.5 h-3.5 text-indigo-500 shrink-0" fill="none"
stroke="currentColor" stroke-width="2" viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" d="M7 21h10a2 2 0
002-2V9.414a1 1 0 00-.293-.707l-5.414-5.414A1 1 0 0012.586 3H7a2 2 0 00-2 2v14a2 2 0 002
2z" />
              </svg>
              <span class="font-medium truncate max-w-
[220px]">${escapeHtml(s.title)}</span>
              <span class="text-[10px] font-semibold text-zinc-400 bg-white px-1.5 py-
0.5 rounded border border-zinc-200/70">${Math.round(s.score * 100)}%</span>
            </button>`
          )
          .join("");
        sourcesHtml = `
          <div class="mt-3.5 pt-3 border-t border-zinc-100">
            <div class="text-[10px] font-semibold text-zinc-400 uppercase tracking-wider
mb-2">Sumber Referensi</div>
            <div class="flex flex-wrap gap-1.5">${items}</div>
          </div>
        `;
      }

      wrapper.innerHTML = `
        <div class="w-7 h-7 rounded-lg bg-indigo-50 border border-indigo-100 text-
indigo-600 flex items-center justify-center shrink-0 mt-0.5">
          <svg class="w-3.5 h-3.5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M8 10h.01M12
10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5
5v-5z" />
          </svg>
        </div>
        <div class="bg-white border border-zinc-200/80 rounded-2xl rounded-tl-md p-4
max-w-[calc(100%-2.5rem)] sm:max-w-2xl min-w-0 shadow-[0_1px_2px_rgba(0,0,0,0.03)]">
          <div class="text-sm leading-relaxed text-zinc-800 break-words [overflow-
wrap:anywhere]">${formatMarkdown(answer)}</div>
          ${sourcesHtml}
        </div>
      `;
      chatBox.appendChild(wrapper);
      chatBox.scrollTop = chatBox.scrollHeight;
    }

    chatForm.addEventListener("submit", async (e) => {
      e.preventDefault();
      const question = questionInput.value.trim();
      if (!question) return;

      const history = getPublicHistory();
      history.push({ role: "user", text: question });
      savePublicHistory(history);
```

```
      appendUserMessage(question);
      questionInput.value = "";
      sendBtn.disabled = true;
      appendLoading();

      try {
        const res = await fetch("/api/chat", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ question, role: "mahasiswa" }),
        });
        const data = await res.json();
        removeLoading();
        const botAnswer = !res.ok ? (data.detail || "Terjadi kesalahan pada server.") :
data.answer;
        const botSources = !res.ok ? [] : (data.sources || []);
        appendBotMessage(botAnswer, botSources);

        const updated = getPublicHistory();
        updated.push({ role: "bot", text: botAnswer, sources: botSources });
        savePublicHistory(updated);
      } catch (err) {
        removeLoading();
        const errText = "Gagal terhubung ke server backend.";
        appendBotMessage(errText);

        const updated = getPublicHistory();
        updated.push({ role: "bot", text: errText, sources: [] });
        savePublicHistory(updated);
      } finally {
        sendBtn.disabled = false;
        questionInput.focus();
      }
    });

    window.openSourceModal = async function (docId, fallbackTitle) {
      const modal = document.getElementById("sourceModal");
      const titleEl = document.getElementById("sourceModalTitle");
      const metaEl = document.getElementById("sourceModalMeta");
      const bodyEl = document.getElementById("sourceModalBody");
      const dlEl = document.getElementById("sourceModalDownload");

      titleEl.textContent = fallbackTitle || "Memuat dokumen...";
      metaEl.textContent = "";
      dlEl.classList.add("hidden");
      bodyEl.innerHTML = `<div class="text-xs text-zinc-500 py-8 text-center">Mengambil
isi dokumen...</div>`;
      modal.classList.remove("hidden");

      try {
        const res = await fetch(`/api/public/documents/${docId}`);
        const data = await res.json();
        if (!res.ok) {
          bodyEl.innerHTML = `<div class="text-xs text-red-600 py-8 text-
center">${escapeHtml(data.detail || "Dokumen tidak ditemukan.")}</div>`;
          return;
        }
        const doc = data.document;
        const content = data.content || "";
        titleEl.textContent = doc.title;
        metaEl.textContent = `Dokumen ${doc.file_type.toUpperCase()}`;
```

```
        dlEl.href = data.url;
        dlEl.classList.remove("hidden");

        bodyEl.innerHTML = !content.trim()
          ? `<div class="text-xs text-zinc-500 py-8 text-center">Isi dokumen
kosong.</div>`
          : `<div class="bg-white border border-zinc-200/80 rounded-xl p-5 shadow-
[0_1px_2px_rgba(0,0,0,0.02)] text-xs leading-relaxed text-zinc-
700">${formatMarkdown(content)}</div>`;
      } catch {
        bodyEl.innerHTML = `<div class="text-xs text-red-600 py-8 text-center">Gagal
memuat isi dokumen.</div>`;
      }
    };

    window.closeSourceModal = function () {
      document.getElementById("sourceModal").classList.add("hidden");
    };

    document.getElementById("sourceModal").addEventListener("click", (e) => {
      if (e.target.id === "sourceModal") closeSourceModal();
    });

    document.addEventListener("keydown", (e) => {
      if (e.key === "Escape") closeSourceModal();
    });

    (function restorePublicChat() {
      const history = getPublicHistory();
      if (history.length > 0) {
        clearChatBtn.classList.remove("hidden");
        history.forEach((item) => {
          if (item.role === "user") appendUserMessage(item.text);
          else appendBotMessage(item.text, item.sources || []);
        });
      }
    })();
  </script>
</body>
</html>
```

## Langkah 12: Membuat Panel Admin (static/admin.html)

File `static/admin.html` menyediakan dashboard admin berbasis JWT untuk mengunggah dokumen RAG, melihat daftar dokumen beserta jumlah chunks, mengaktifkan atau menonaktifkan dokumen tanpa menghapusnya, membuka popup **Lihat Chunks**, menguji chatbot melalui popup **Buka Chatbot** (lengkap dengan riwayat multi-sesi), dan memantau sisa kuota RPD/TPM melalui popup **Kuota Groq API**.

**static/admin.html**

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Login Admin — Chatbot RAG</title>
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
```

```
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet" />
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          fontFamily: { sans: ['Inter', 'system-ui', 'sans-serif'] }
        }
      }
    }
  </script>
  <style>
    .bg-grid {
      background-image: radial-gradient(rgba(99, 102, 241, 0.08) 1px, transparent 1px);
      background-size: 24px 24px;
    }
  </style>
</head>
<body class="bg-[#FAFAFB] text-zinc-900 font-sans antialiased min-h-screen">

  <!-- Toast Notification -->
  <div id="toast" class="hidden fixed bottom-5 right-5 z-50 flex items-center gap-2 bg-
zinc-900 text-white text-xs font-medium px-4 py-2.5 rounded-xl shadow-lg">
    <span class="w-1.5 h-1.5 rounded-full bg-emerald-400"></span>
    <span id="toastText"></span>
  </div>

  <!-- Chunks Modal Popup -->
  <div id="chunksModal" class="hidden fixed inset-0 z-50 bg-zinc-900/50 backdrop-blur-xs
flex items-center justify-center p-4">
    <div class="bg-white border border-zinc-200 rounded-2xl shadow-2xl w-full max-w-2xl
max-h-[82vh] flex flex-col overflow-hidden">
      <div class="px-5 py-4 border-b border-zinc-100 flex items-center justify-between
gap-3">
        <div class="min-w-0">
          <h3 id="modalDocTitle" class="text-sm font-semibold text-zinc-900
truncate">Detail Chunks</h3>
          <p id="modalDocMeta" class="text-xs text-zinc-500 mt-0.5"></p>
        </div>
        <button type="button" onclick="closeChunksModal()" class="text-zinc-400
hover:text-zinc-700 p-1.5 rounded-lg hover:bg-zinc-100 transition cursor-pointer">
          <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M6 18L18 6M6 6l12
12" />
          </svg>
        </button>
      </div>
      <div id="modalChunksList" class="p-5 overflow-y-auto overflow-x-hidden space-y-3
bg-zinc-50/50 flex-1"></div>
      <div class="px-5 py-3 border-t border-zinc-100 bg-white flex justify-end">
        <button type="button" onclick="closeChunksModal()" class="text-xs font-medium
text-zinc-700 bg-zinc-100 hover:bg-zinc-200 px-4 py-2 rounded-xl transition cursor-
pointer">
          Tutup
        </button>
      </div>
    </div>
  </div>
```

```
  <!-- Chatbot Modal Popup -->
  <div id="chatModal" class="hidden fixed inset-0 z-50 bg-zinc-900/50 backdrop-blur-xs
flex items-center justify-center p-4">
    <div class="bg-white border border-zinc-200 rounded-2xl shadow-2xl w-full max-w-2xl
h-[82vh] flex flex-col overflow-hidden">
      <div class="px-5 py-3.5 border-b border-zinc-100 flex items-center justify-between
gap-3">
        <div class="flex items-center gap-2.5 min-w-0">
          <div class="w-8 h-8 rounded-xl bg-gradient-to-br from-indigo-600 to-violet-600
text-white flex items-center justify-center shadow-xs shrink-0">
            <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M8 10h.01M12
10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5
5v-5z" />
            </svg>
          </div>
          <div class="min-w-0">
            <h3 class="text-sm font-semibold text-zinc-900 truncate">Chatbot RAG</h3>
            <p class="text-[11px] text-zinc-500">Uji coba asisten dokumen akademik</p>
          </div>
        </div>
        <div class="flex items-center gap-1.5 shrink-0">
          <button type="button" onclick="startNewAdminChat()" class="text-[11px] font-
medium text-indigo-600 bg-indigo-50 hover:bg-indigo-100 px-2.5 py-1.5 rounded-lg
transition cursor-pointer">
            + Chat Baru
          </button>
          <button type="button" id="toggleHistoryBtn" onclick="toggleAdminHistoryView()"
class="text-[11px] font-medium text-zinc-600 hover:text-zinc-900 bg-zinc-100 hover:bg-
zinc-200/80 px-2.5 py-1.5 rounded-lg transition cursor-pointer">
            Riwayat
          </button>
          <button type="button" onclick="closeChatModal()" class="text-zinc-400
hover:text-zinc-700 p-1.5 rounded-lg hover:bg-zinc-100 transition cursor-pointer">
            <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M6 18L18 6M6 6l12
12" />
            </svg>
          </button>
        </div>
      </div>

      <!-- Riwayat Chat Panel (Hidden by default) -->
      <div id="adminHistoryPanel" class="hidden flex-1 p-5 overflow-y-auto bg-zinc-50/50
space-y-2.5"></div>

      <div id="adminChatBox" class="flex-1 p-5 overflow-y-auto overflow-x-hidden space-
y-4 bg-zinc-50/50"></div>

      <div id="adminChatFooter" class="p-3.5 border-t border-zinc-100 bg-white">
        <form id="adminChatForm" class="bg-zinc-50 border border-zinc-200/90 rounded-xl
p-1.5 flex items-center gap-2 focus-within:bg-white focus-within:ring-2 focus-
within:ring-indigo-500/20 focus-within:border-indigo-400 transition">
          <input
            id="adminQuestionInput"
            type="text"
            placeholder="Tulis pertanyaan untuk menguji chatbot..."
            autocomplete="off"
            class="flex-1 px-3 py-1.5 text-sm text-zinc-900 placeholder:text-zinc-400
focus:outline-none bg-transparent"
```

```
            required
          />
          <button
            id="adminSendBtn"
            type="submit"
            class="inline-flex items-center gap-1.5 bg-indigo-600 hover:bg-indigo-700
disabled:opacity-40 text-white text-xs font-medium px-3.5 py-2 rounded-lg shadow-xs
transition cursor-pointer"
          >
            <span>Kirim</span>
            <svg class="w-3.5 h-3.5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M22 2L11 13M22 2l-
7 20-4-9-9-4 20-7z" />
            </svg>
          </button>
        </form>
      </div>
    </div>
  </div>

  <!-- Modal Tracking Kuota Groq API -->
  <div id="quotaModal" class="hidden fixed inset-0 z-50 bg-zinc-950/40 backdrop-blur-
[2px] flex items-center justify-center p-4">
    <div class="bg-white border border-zinc-200 rounded-2xl shadow-2xl w-full max-w-2xl
max-h-[85vh] flex flex-col overflow-hidden">
      <div class="px-5 py-4 border-b border-zinc-100 flex items-center justify-between
gap-3">
        <div class="flex items-center gap-2.5 min-w-0">
          <div class="w-8 h-8 rounded-xl bg-gradient-to-br from-indigo-600 to-violet-600
text-white flex items-center justify-center shrink-0 shadow-xs">
            <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M9 19v-6a2 2 0 00-
2-2H5a2 2 0 00-2 2v6a2 2 0 002 2h2a2 2 0 002-2zm0 0V9a2 2 0 012-2h2a2 2 0 012 2v10m-6
0a2 2 0 002 2h2a2 2 0 002-2m0 0V5a2 2 0 012-2h2a2 2 0 012 2v14a2 2 0 01-2 2h-2a2 2 0 01-
2-2z" />
            </svg>
          </div>
          <div class="min-w-0">
            <h3 class="text-sm font-semibold text-zinc-900">Tracking Kuota Groq API</h3>
            <p id="quotaModelInfo" class="text-xs text-zinc-500 mt-0.5 truncate">Memuat
status...</p>
          </div>
        </div>
        <div class="flex items-center gap-1.5 shrink-0">
          <button type="button" onclick="loadGroqQuota()" class="text-xs font-medium
text-indigo-600 bg-indigo-50 hover:bg-indigo-100 px-2.5 py-1.5 rounded-lg transition
cursor-pointer">
            Refresh
          </button>
          <button type="button" onclick="closeQuotaModal()" class="text-zinc-400
hover:text-zinc-700 p-1.5 rounded-lg hover:bg-zinc-100 transition cursor-pointer">
            <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M6 18L18 6M6 6l12
12" />
            </svg>
          </button>
        </div>
      </div>
```

```
      <div id="quotaContent" class="p-5 overflow-y-auto space-y-4 bg-zinc-50/40">
        <div class="text-center text-xs text-zinc-400 py-8">Memuat data kuota...</div>
      </div>
    </div>
  </div>

  <!-- Login Section -->
  <section id="loginSection" class="min-h-screen bg-grid flex items-center justify-
center p-4">
    <div class="bg-white border border-zinc-200/80 rounded-2xl shadow-xl shadow-zinc-
200/40 w-full max-w-sm p-7">
      <div class="flex items-center gap-3 mb-5">
        <div class="w-10 h-10 rounded-xl bg-gradient-to-br from-indigo-600 to-violet-600
text-white flex items-center justify-center shadow-sm shadow-indigo-500/25">
          <svg class="w-5 h-5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M12 15v2m-6 4h12a2 2
0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z" />
          </svg>
        </div>
        <div>
          <h1 class="text-base font-semibold tracking-tight text-zinc-900">Login
Admin</h1>
          <p class="text-xs text-zinc-500">Chatbot RAG</p>
        </div>
      </div>

      <div id="loginError" class="hidden mb-4 text-xs bg-red-50 text-red-700 border
border-red-200 rounded-xl p-3"></div>

      <form id="loginForm" class="space-y-3.5">
        <div>
          <label class="block text-xs font-medium text-zinc-700 mb-1.5">Username</label>
          <input id="loginUsername" type="text" placeholder="admin" required
            class="w-full border border-zinc-200 rounded-xl px-3.5 py-2.5 text-sm bg-
zinc-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500 transition" />
        </div>
        <div>
          <label class="block text-xs font-medium text-zinc-700 mb-1.5">Password</label>
          <input id="loginPassword" type="password" placeholder="••••••••" required
            class="w-full border border-zinc-200 rounded-xl px-3.5 py-2.5 text-sm bg-
zinc-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500 transition" />
        </div>
        <button type="submit" class="w-full bg-indigo-600 hover:bg-indigo-700 text-white
text-sm font-medium py-2.5 rounded-xl shadow-xs transition cursor-pointer">
          Masuk ke Dashboard
        </button>
      </form>

      <div class="mt-4 bg-zinc-50 border border-zinc-200/70 rounded-xl px-3 py-2 text-
[11px] text-zinc-500 flex items-center justify-between">
        <span>Akun default:</span>
        <code class="text-zinc-700 font-medium">admin / admin123</code>
      </div>

      <div class="mt-5 pt-4 border-t border-zinc-100 text-center">
        <a href="/" class="text-xs text-zinc-500 hover:text-indigo-600 transition font-
medium">Kembali ke Halaman Chatbot</a>
      </div>
    </div>
```

```
  </section>

  <!-- Dashboard Layout (Sidebar + Main) -->
  <section id="dashboardSection" class="hidden min-h-screen flex flex-col md:flex-row">
    <!-- Sidebar -->
    <aside class="w-full md:w-64 bg-white border-b md:border-b-0 md:border-r border-
zinc-200/80 flex flex-col shrink-0">
      <div class="h-14 px-5 flex items-center gap-2.5 border-b border-zinc-100">
        <div class="w-7 h-7 rounded-lg bg-gradient-to-br from-indigo-600 to-violet-600
text-white flex items-center justify-center shadow-xs">
          <svg class="w-3.5 h-3.5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M12 6.253v13m0-
13C10.832 5.477 9.246 5 7.5 5S4.168 5.477 3 6.253v13C4.168 18.477 5.754 18 7.5
18s3.332.477 4.5 1.253m0-13C13.168 5.477 14.754 5 16.5 5c1.747 0 3.332.477 4.5
1.253v13C19.832 18.477 18.247 18 16.5 18c-1.746 0-3.332.477-4.5 1.253" />
          </svg>
        </div>
        <span class="text-sm font-semibold tracking-tight text-zinc-900">Admin
Panel</span>
      </div>

      <nav class="p-3 space-y-1 flex-1">
        <div class="px-2.5 py-1.5 text-[10px] font-semibold uppercase tracking-wider
text-zinc-400">Workspace</div>
        <a href="#knowledge" class="flex items-center gap-2.5 px-3 py-2 rounded-xl text-
xs font-medium bg-indigo-50/80 text-indigo-700">
          <svg class="w-4 h-4" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M19 11H5m14 0a2 2 0
012 2v6a2 2 0 01-2 2H5a2 2 0 01-2-2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-
2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7 7h10" />
          </svg>
          Knowledge Base
        </a>
        <button type="button" onclick="openChatModal()" class="w-full flex items-center
gap-2.5 px-3 py-2 rounded-xl text-xs font-medium text-zinc-600 hover:bg-zinc-100/80
hover:text-zinc-900 transition cursor-pointer">
          <svg class="w-4 h-4 text-zinc-400" fill="none" stroke="currentColor" stroke-
width="2" viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M8 10h.01M12
10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5
5v-5z" />
          </svg>
          Buka Chatbot
        </button>
        <button type="button" onclick="openQuotaModal()" class="w-full flex items-center
gap-2.5 px-3 py-2 rounded-xl text-xs font-medium text-zinc-600 hover:bg-zinc-100/80
hover:text-zinc-900 transition cursor-pointer">
          <svg class="w-4 h-4 text-zinc-400" fill="none" stroke="currentColor" stroke-
width="2" viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" d="M9 19v-6a2 2 0 00-2-
2H5a2 2 0 00-2 2v6a2 2 0 002 2h2a2 2 0 002-2zm0 0V9a2 2 0 012-2h2a2 2 0 012 2v10m-6 0a2
2 0 002 2h2a2 2 0 002-2m0 0V5a2 2 0 012-2h2a2 2 0 012 2v14a2 2 0 01-2 2h-2a2 2 0 01-2-
2z" />
          </svg>
          Kuota Groq API
        </button>
      </nav>

      <div class="p-3 border-t border-zinc-100 flex items-center justify-between gap-2">
        <div class="flex items-center gap-2 min-w-0">
```

```
          <div class="w-7 h-7 rounded-full bg-zinc-900 text-white text-[11px] font-
semibold flex items-center justify-center shrink-0">A</div>
          <div class="min-w-0">
            <div class="text-xs font-medium text-zinc-800 truncate">Admin SSC</div>
            <div class="text-[10px] text-emerald-600 font-medium">● Online</div>
          </div>
        </div>
        <button id="logoutBtn" class="text-xs font-medium text-zinc-500 hover:text-red-
600 px-2 py-1 rounded-lg hover:bg-red-50 transition cursor-pointer">
          Logout
        </button>
      </div>
    </aside>

    <!-- Main Content -->
    <main class="flex-1 p-5 sm:p-8 max-w-6xl w-full space-y-6">
      <div>
        <h1 class="text-xl font-semibold tracking-tight text-zinc-900">Knowledge
Base</h1>
        <p class="text-xs text-zinc-500 mt-0.5">Unggah dan kelola dokumen referensi
untuk Chatbot RAG.</p>
      </div>

      <!-- KPI Cards (2 Columns) -->
      <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
        <div class="bg-white border border-zinc-200/80 rounded-2xl p-4 shadow-
[0_1px_2px_rgba(0,0,0,0.03)] flex items-center justify-between">
          <div>
            <div class="text-xs font-medium text-zinc-500">Total Dokumen</div>
            <div id="statDocs" class="text-2xl font-semibold text-zinc-900 mt-1">0</div>
          </div>
          <div class="w-10 h-10 rounded-xl bg-emerald-50 text-emerald-600 flex items-
center justify-center">
            <svg class="w-5 h-5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M9 12h6m-6 4h6m2
5H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2
2 0 01-2 2z" />
            </svg>
          </div>
        </div>

        <div class="bg-white border border-zinc-200/80 rounded-2xl p-4 shadow-
[0_1px_2px_rgba(0,0,0,0.03)] flex items-center justify-between">
          <div>
            <div class="text-xs font-medium text-zinc-500">Chunks Terindeks</div>
            <div id="statChunks" class="text-2xl font-semibold text-zinc-900 mt-
1">0</div>
          </div>
          <div class="w-10 h-10 rounded-xl bg-indigo-50 text-indigo-600 flex items-
center justify-center">
            <svg class="w-5 h-5" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
              <path stroke-linecap="round" stroke-linejoin="round" d="M4 7v10c0 2.21
3.582 4 8 4s8-1.79 8-4V7M4 7c0 2.21 3.582 4 8 4s8-1.79 8-4M4 7c0-2.21 3.582-4 8-4s8 1.79
8 4" />
            </svg>
          </div>
        </div>
      </div>

      <!-- Upload & Documents Grid -->
```

```
      <div class="grid grid-cols-1 lg:grid-cols-12 gap-6">
        <!-- Left: Upload Form -->
        <div class="lg:col-span-5 bg-white border border-zinc-200/80 rounded-2xl p-5
shadow-[0_1px_2px_rgba(0,0,0,0.03)] h-fit">
          <h2 class="text-sm font-semibold text-zinc-900">Upload Dokumen RAG</h2>
          <p class="text-xs text-zinc-500 mt-0.5 mb-4">Dokumen akan diekstrak, di-
chunking, dan diindeks ke database.</p>

          <div id="uploadMsg" class="hidden mb-4 text-xs rounded-xl p-3"></div>

          <form id="uploadForm" class="space-y-4">
            <div>
              <label class="block text-xs font-medium text-zinc-700 mb-1.5">Judul
Dokumen (opsional)</label>
              <input id="docTitle" type="text" placeholder="Misal: Pedoman TAK 2025"
                class="w-full border border-zinc-200 rounded-xl px-3.5 py-2 text-sm bg-
zinc-50/50 focus:bg-white focus:outline-none focus:ring-2 focus:ring-indigo-500/20
focus:border-indigo-500 transition" />
            </div>

            <!-- Custom Dropzone -->
            <div>
              <label class="block text-xs font-medium text-zinc-700 mb-1.5">File Dokumen
(PDF, DOCX, TXT)</label>
              <label for="docFile" class="flex flex-col items-center justify-center
border-2 border-dashed border-zinc-200 hover:border-indigo-400 bg-zinc-50/60 hover:bg-
indigo-50/30 rounded-xl p-5 cursor-pointer transition text-center">
                <svg class="w-6 h-6 text-zinc-400 mb-1.5" fill="none"
stroke="currentColor" stroke-width="1.8" viewBox="0 0 24 24">
                  <path stroke-linecap="round" stroke-linejoin="round" d="M7 16a4 4 0
01-.88-7.903A5 5 0 1115.9 6L16 6a5 5 0 011 9.9M15 13l-3-3m0 0l-3 3m3-3v12" />
                </svg>
                <span id="fileLabel" class="text-xs font-medium text-zinc-600">Klik
untuk pilih file PDF, DOCX, XLSX, CSV, atau TXT</span>
                <input id="docFile" type="file" accept=".pdf,.docx,.xlsx,.txt,.md,.csv"
required class="hidden" />
              </label>
            </div>

            <button id="uploadBtn" type="submit" class="w-full bg-indigo-600 hover:bg-
indigo-700 disabled:opacity-50 text-white text-xs font-medium px-5 py-2.5 rounded-xl
shadow-xs transition cursor-pointer">
              Upload &amp;amp; Indeks Dokumen
            </button>
          </form>
        </div>

        <!-- Right: Document List -->
        <div class="lg:col-span-7 bg-white border border-zinc-200/80 rounded-2xl p-5
shadow-[0_1px_2px_rgba(0,0,0,0.03)]">
          <h2 class="text-sm font-semibold text-zinc-900 mb-3">Daftar Dokumen
Terindeks</h2>
          <div id="docList" class="divide-y divide-zinc-100"></div>
        </div>
      </div>
    </main>
  </section>

  <script>
    const loginSection = document.getElementById("loginSection");
    const dashboardSection = document.getElementById("dashboardSection");
    const loginForm = document.getElementById("loginForm");
```

```
    const loginError = document.getElementById("loginError");
    const logoutBtn = document.getElementById("logoutBtn");
    const docList = document.getElementById("docList");
    const uploadMsg = document.getElementById("uploadMsg");
    const docFileInput = document.getElementById("docFile");
    const fileLabel = document.getElementById("fileLabel");
    const toast = document.getElementById("toast");
    const toastText = document.getElementById("toastText");
    let defaultRoleIds = [];

    docFileInput.addEventListener("change", () => {
      if (docFileInput.files && docFileInput.files[0]) {
        const f = docFileInput.files[0];
        fileLabel.textContent = `${f.name} (${Math.round(f.size / 1024)} KB)`;
        fileLabel.classList.add("text-indigo-700");
      } else {
        fileLabel.textContent = "Klik untuk pilih file PDF, DOCX, atau TXT";
        fileLabel.classList.remove("text-indigo-700");
      }
    });

    function getToken() {
      return localStorage.getItem("admin_token") || "";
    }

    function showToast(msg) {
      toastText.textContent = msg;
      toast.classList.remove("hidden");
      setTimeout(() => toast.classList.add("hidden"), 2800);
    }

    function escapeHtml(str) {
      return String(str)
        .replace(/&/g, "&amp;")
        .replace(/</g, "&lt;")
        .replace(/>/g, "&gt;")
        .replace(/"/g, "&quot;");
    }

    function formatMarkdown(text) {
      const escaped = escapeHtml(text || "");
      const lines = escaped.split(/\r?\n/);
      let html = "";
      let inUl = false;
      let inOl = false;

      function cleanLatex(expr) {
        return expr
          .replace(/\\left\s*([(\[{|])/g, "$1")
          .replace(/\\right\s*([)\]}|])/g, "$1")
          .replace(/\\frac\s*\{([^{}]+)\}\s*\{([^{}]+)\}/g, "$1 / $2")
          .replace(/\\times/g, "×")
          .replace(/\\div/g, "÷")
          .replace(/\\cdot/g, "·")
          .replace(/\\pm/g, "±")
          .replace(/\\leq/g, "≤")
          .replace(/\\geq/g, "≥")
          .replace(/\\neq/g, "≠")
          .replace(/\\approx/g, "≈")
          .replace(/\\%/g, "%")
          .replace(/_\{([^{}]+)\}/g, "$1")
          .replace(/\^\{([^{}]+)\}/g, "^$1")
```

```
          .replace(/\(\s+/g, "(")
          .replace(/\s+\)/g, ")")
          .replace(/\s{2,}/g, " ")
          .trim();
      }

      function inlineFormat(s) {
        const links = [];
        const stashLink = (href, label, external = true) => {
          const idx = links.length;
          const attrs = external ? ' target="_blank" rel="noopener noreferrer"' : "";
          links.push(`<a href="${href}"${attrs} class="text-indigo-600 hover:underline
break-all">${label}</a>`);
          return `\u0000LINK${idx}\u0000`;
        };

        let out = s
          .replace(/\[([^\]]+)\]\(((?:https?:\/\/|mailto:)[^\s)]+)\)/g, (_, label, url)
=>
            stashLink(url, label, url.startsWith("http"))
          )
          .replace(/(^|[\s(>])(https?:\/\/[^\s<)]+)/g, (_, pre, url) => {
            const cleanUrl = url.replace(/[.,;:!?]+$/, "");
            const trail = url.slice(cleanUrl.length);
            return pre + stashLink(cleanUrl, cleanUrl, true) + trail;
          })
          .replace(/(^|[\s(>])([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})/g, (_,
pre, email) =>
            pre + stashLink(`mailto:${email}`, email, false)
          )
          .replace(/(^|[\s(>])((?:linktr\.ee|(?:[a-zA-Z0-9-
]+\.)+(?:ac\.id|id|com|org|edu|net))(?:\/[^\s<)]*)?)/g, (_, pre, domainUrl) => {
            const cleanUrl = domainUrl.replace(/[.,;:!?]+$/, "");
            const trail = domainUrl.slice(cleanUrl.length);
            return pre + stashLink(`https://${cleanUrl}`, cleanUrl, true) + trail;
          })
          .replace(/\*\*([^*]+)\*\*/g, '<strong class="font-semibold text-zinc-
900">$1</strong>')
          .replace(/(^|[^\*])\*([^*\n]+)\*/g, '$1$2')
          .replace(/`([^`]+)`/g, '<code class="bg-zinc-100 text-zinc-800 px-1 py-0.5
rounded text-[11px]">$1</code>')
          .replace(/\$\$([^$]+)\$\$/g, (_, math) => `<span class="font-mono text-[11px]
font-medium text-zinc-900 bg-zinc-100 px-1.5 py-0.5
rounded">${cleanLatex(math)}</span>`)
          .replace(/\$([^$\n]+)\$/g, (_, math) => `<span class="font-mono text-[11px]
font-medium text-zinc-900 bg-zinc-100 px-1 py-0.5 rounded">${cleanLatex(math)}</span>`)
          .replace(/\\\((.+?)\\\)/g, (_, math) => `<span class="font-mono text-[11px]
font-medium text-zinc-900 bg-zinc-100 px-1 py-0.5 rounded">${cleanLatex(math)}</span>`);

        return out.replace(/\u0000LINK(\d+)\u0000/g, (_, i) => links[Number(i)]);
      }

      function closeLists() {
        if (inUl) { html += "</ul>"; inUl = false; }
        if (inOl) { html += "</ol>"; inOl = false; }
      }

      function isTableRow(line) {
        return line.startsWith("|") && line.endsWith("|") && line.length > 2;
      }

      function isTableSeparator(line) {
```

```
        return /^\|[\s:\-\|]+\|$/.test(line);
      }

      function parseTableCells(line) {
        return line.slice(1, -1).split("|").map((c) => c.trim());
      }

      for (let i = 0; i < lines.length; i++) {
        const line = lines[i].trim();
        if (!line) {
          closeLists();
          continue;
        }

        if (/^(-{3,}|\*{3,})$/.test(line)) {
          closeLists();
          html += '<hr class="my-2.5 border-zinc-200/80" />';
          continue;
        }

        const blockMathMatch = line.match(/^(?:\$\$(.+)\$\$|\\\[(.+)\\\])$/);
        if (blockMathMatch) {
          closeLists();
          const expr = cleanLatex(blockMathMatch[1] || blockMathMatch[2] || "");
          html += `<div class="my-2 px-3 py-2 rounded-xl bg-zinc-50 border border-zinc-
200/80 font-mono text-xs font-medium text-zinc-900 text-center overflow-x-
auto">${expr}</div>`;
          continue;
        }

        if (isTableRow(line) && i + 1 < lines.length && isTableSeparator(lines[i +
1].trim())) {
          closeLists();
          const headers = parseTableCells(line);
          i += 1;
          let tableHtml = '<div class="w-full max-w-full overflow-x-auto my-2 rounded-xl
border border-zinc-200/80"><table class="w-full text-left border-collapse text-xs break-
normal [overflow-wrap:normal]">';
          tableHtml += '<thead class="bg-zinc-50 border-b border-zinc-200/80"><tr>';
          headers.forEach((h) => {
            tableHtml += `<th class="px-2.5 py-1.5 font-semibold text-zinc-800 align-
bottom leading-snug">${inlineFormat(h)}</th>`;
          });
          tableHtml += '</tr></thead><tbody class="divide-y divide-zinc-100 bg-white">';
          while (i + 1 < lines.length && isTableRow(lines[i + 1].trim())) {
            i += 1;
            const cells = parseTableCells(lines[i].trim());
            tableHtml += '<tr>';
            headers.forEach((_, colIdx) => {
              tableHtml += `<td class="px-2.5 py-1.5 text-zinc-700 align-top whitespace-
nowrap sm:whitespace-normal">${inlineFormat(cells[colIdx] || "")}</td>`;
            });
            tableHtml += '</tr>';
          }
          tableHtml += '</tbody></table></div>';
          html += tableHtml;
          continue;
        }

        const ulMatch = line.match(/^[\*\-]\s+(.*)$/);
        const olMatch = line.match(/^(\d+)\.\s+(.*)$/);
        const hMatch = line.match(/^(#{1,3})\s+(.*)$/);
```

```
        if (ulMatch) {
          if (inOl) { html += "</ol>"; inOl = false; }
          if (!inUl) { html += '<ul class="list-disc pl-4 space-y-1 my-1.5">'; inUl =
true; }
          html += `<li>${inlineFormat(ulMatch[1])}</li>`;
        } else if (olMatch) {
          if (inUl) { html += "</ul>"; inUl = false; }
          if (!inOl) { html += '<ol class="list-decimal pl-4 space-y-1 my-1.5">'; inOl =
true; }
          html += `<li value="${olMatch[1]}">${inlineFormat(olMatch[2])}</li>`;
        } else if (hMatch) {
          closeLists();
          const level = hMatch[1].length;
          const cls = level === 1 ? "text-sm font-semibold text-zinc-900 mt-2.5 mb-1" :
"text-xs font-semibold text-zinc-900 mt-2 mb-1";
          html += `<div class="${cls}">${inlineFormat(hMatch[2])}</div>`;
        } else {
          closeLists();
          html += `<p class="my-1 first:mt-0 last:mb-0">${inlineFormat(line)}</p>`;
        }
      }
      closeLists();
      return html;
    }

    async function checkAuth() {
      const token = getToken();
      if (!token) return showLogin();
      try {
        const res = await fetch("/api/auth/me", { headers: { Authorization: `Bearer
${token}` } });
        if (!res.ok) throw new Error();
        showDashboard();
      } catch {
        localStorage.removeItem("admin_token");
        showLogin();
      }
    }

    function showLogin() {
      loginSection.classList.remove("hidden");
      dashboardSection.classList.add("hidden");
    }

    function showDashboard() {
      loginSection.classList.add("hidden");
      dashboardSection.classList.remove("hidden");
      loadRolesForUpload();
      loadDocuments();
    }

    loginForm.addEventListener("submit", async (e) => {
      e.preventDefault();
      loginError.classList.add("hidden");
      const username = document.getElementById("loginUsername").value.trim();
      const password = document.getElementById("loginPassword").value;

      const res = await fetch("/api/auth/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username, password }),
```

```
      });
      const data = await res.json();
      if (!res.ok) {
        loginError.textContent = data.detail || "Login gagal.";
        loginError.classList.remove("hidden");
        return;
      }
      localStorage.setItem("admin_token", data.token);
      showDashboard();
    });

    logoutBtn.addEventListener("click", () => {
      localStorage.removeItem("admin_token");
      window.location.href = "/";
    });

    async function loadRolesForUpload() {
      const res = await fetch("/api/roles");
      const data = await res.json();
      const roles = data.roles || [];
      defaultRoleIds = roles.map((r) => r.id);
    }

    async function loadDocuments() {
      const res = await fetch("/api/documents", {
        headers: { Authorization: `Bearer ${getToken()}` },
      });
      const data = await res.json();
      const docs = data.documents || [];
      document.getElementById("statDocs").textContent = docs.length;
      document.getElementById("statChunks").textContent = docs.reduce((acc, d) => acc +
(d.total_chunks || 0), 0);

      docList.innerHTML = docs.length === 0
        ? `<div class="py-12 text-center">
             <div class="w-10 h-10 rounded-xl bg-zinc-100 text-zinc-400 flex items-
center justify-center mx-auto mb-2.5">
               <svg class="w-5 h-5" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                 <path stroke-linecap="round" stroke-linejoin="round" d="M9 12h6m-6
4h6m2 5H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0
01.293.707V19a2 2 0 01-2 2z" />
               </svg>
             </div>
             <p class="text-xs font-medium text-zinc-500">Belum ada dokumen yang
diunggah</p>
             <p class="text-[11px] text-zinc-400 mt-0.5">Unggah file PDF, DOCX, atau TXT
di panel sebelah kiri.</p>
           </div>`
        : docs.map((d) => {
            const isActive = d.is_active !== 0;
            const fileBadge = `<span class="text-[11px] bg-indigo-50 text-indigo-700
border border-indigo-100 px-2 py-0.5 rounded-md font-
medium">${escapeHtml(d.file_type.toUpperCase())}</span>`;
            const statusBadge = isActive
              ? ""
              : `<span class="text-[11px] bg-amber-50 text-amber-700 border border-
amber-200/80 px-2 py-0.5 rounded-md font-medium">Nonaktif</span>`;
            const toggleLabel = isActive ? "Nonaktifkan" : "Aktifkan";
            const toggleCls = isActive
              ? "text-xs text-amber-600 hover:bg-amber-50 font-medium px-2.5 py-1.5
rounded-lg transition cursor-pointer"
```

```
              : "text-xs text-emerald-600 hover:bg-emerald-50 font-medium px-2.5 py-1.5
rounded-lg transition cursor-pointer";
            return `
              <div class="flex flex-col sm:flex-row sm:items-center justify-between gap-
3 py-3.5 first:pt-0 last:pb-0 ${isActive ? "" : "opacity-65"}">
                <div class="min-w-0">
                  <button type="button" onclick="viewChunks(${d.id})" class="text-left
text-sm font-medium text-zinc-900 hover:text-indigo-600 transition cursor-pointer break-
words">
                    ${escapeHtml(d.title)}
                  </button>
                  <div class="text-xs text-zinc-500 mt-0.5">${d.total_chunks}
chunks</div>
                  <div class="flex flex-wrap gap-1 mt-
1.5">${fileBadge}${statusBadge}</div>
                </div>
                <div class="flex items-center gap-1.5 shrink-0">
                  <button type="button" onclick="viewChunks(${d.id})"
                    class="text-xs text-indigo-600 hover:bg-indigo-50 font-medium px-2.5
py-1.5 rounded-lg transition cursor-pointer">
                    Lihat Chunks
                  </button>
                  <button type="button" onclick="toggleDocument(${d.id})"
                    class="${toggleCls}">
                    ${toggleLabel}
                  </button>
                  <button type="button" onclick="deleteDocument(${d.id},
'${escapeHtml(d.title)}')"
                    class="text-xs text-red-600 hover:bg-red-50 font-medium px-2.5 py-
1.5 rounded-lg transition cursor-pointer">
                    Hapus
                  </button>
                </div>
              </div>
            `;
          }).join("");
    }

    document.getElementById("uploadForm").addEventListener("submit", async (e) => {
      e.preventDefault();
      uploadMsg.classList.add("hidden");

      if (defaultRoleIds.length === 0) {
        await loadRolesForUpload();
      }

      const formData = new FormData();
      formData.append("file", docFileInput.files[0]);
      formData.append("title", document.getElementById("docTitle").value.trim());
      formData.append("role_ids", JSON.stringify(defaultRoleIds));

      const btn = document.getElementById("uploadBtn");
      btn.disabled = true;
      btn.textContent = "Memproses...";

      try {
        const res = await fetch("/api/documents/upload", {
          method: "POST",
          headers: { Authorization: `Bearer ${getToken()}` },
          body: formData,
        });
        const data = await res.json();
```

```
        if (!res.ok) {
          uploadMsg.className = "mb-4 text-xs rounded-xl p-3 bg-red-50 text-red-700
border border-red-200";
          uploadMsg.textContent = data.detail || "Gagal mengunggah dokumen.";
        } else {
          uploadMsg.className = "mb-4 text-xs rounded-xl p-3 bg-emerald-50 text-emerald-
700 border border-emerald-200";
          uploadMsg.textContent = data.message;
          document.getElementById("uploadForm").reset();
          loadRolesForUpload();
          fileLabel.textContent = "Klik untuk pilih file PDF, DOCX, atau TXT";
          fileLabel.classList.remove("text-indigo-700");
          loadDocuments();
        }
        uploadMsg.classList.remove("hidden");
      } finally {
        btn.disabled = false;
        btn.textContent = "Upload & Indeks Dokumen";
      }
    });

    window.viewChunks = async function (docId) {
      const modal = document.getElementById("chunksModal");
      const titleEl = document.getElementById("modalDocTitle");
      const metaEl = document.getElementById("modalDocMeta");
      const listEl = document.getElementById("modalChunksList");

      titleEl.textContent = "Memuat chunks...";
      metaEl.textContent = "";
      listEl.innerHTML = `<div class="text-xs text-zinc-500 py-8 text-center">Mengambil
data chunk...</div>`;
      modal.classList.remove("hidden");

      try {
        const res = await fetch(`/api/documents/${docId}/chunks`, {
          headers: { Authorization: `Bearer ${getToken()}` },
        });
        const data = await res.json();
        if (!res.ok) {
          listEl.innerHTML = `<div class="text-xs text-red-600 py-8 text-
center">${escapeHtml(data.detail || "Gagal memuat chunks.")}</div>`;
          return;
        }
        const doc = data.document;
        const chunks = data.chunks || [];
        titleEl.textContent = doc.title;
        metaEl.textContent = `${doc.file_type.toUpperCase()} · ${chunks.length} chunks`;
        listEl.innerHTML = chunks.length === 0
          ? `<div class="text-xs text-zinc-500 py-8 text-center">Tidak ada chunk pada
dokumen ini.</div>`
          : chunks.map((c) => `
              <div class="bg-white border border-zinc-200/80 rounded-xl p-3.5 shadow-
[0_1px_2px_rgba(0,0,0,0.02)] min-w-0 overflow-hidden">
                <div class="text-[11px] font-semibold text-indigo-600 mb-1.5">Chunk
#${c.chunk_index + 1}</div>
                <div class="text-xs text-zinc-700 leading-relaxed break-words [overflow-
wrap:anywhere]">${formatMarkdown(c.text)}</div>
              </div>
            `).join("");
      } catch {
        listEl.innerHTML = `<div class="text-xs text-red-600 py-8 text-center">Gagal
terhubung ke server.</div>`;
```

```
    }
};

window.closeChunksModal = function () {
  document.getElementById("chunksModal").classList.add("hidden");
};

document.getElementById("chunksModal").addEventListener("click", (e) => {
  if (e.target.id === "chunksModal") closeChunksModal();
});

document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") {
    closeChunksModal();
    closeChatModal();
  }
});

// ---------- Admin Chatbot Popup & Multi-Session History ----------
const CHAT_SESSIONS_KEY = "admin_chat_sessions";
const ACTIVE_SESSION_KEY = "admin_active_session_id";
const adminChatBox = document.getElementById("adminChatBox");
const adminHistoryPanel = document.getElementById("adminHistoryPanel");
const adminChatFooter = document.getElementById("adminChatFooter");
const toggleHistoryBtn = document.getElementById("toggleHistoryBtn");
const adminChatForm = document.getElementById("adminChatForm");
const adminQuestionInput = document.getElementById("adminQuestionInput");
const adminSendBtn = document.getElementById("adminSendBtn");
let showingHistory = false;

function getAdminSessions() {
  try {
    return JSON.parse(localStorage.getItem(CHAT_SESSIONS_KEY) || "[]");
  } catch {
    return [];
  }
}

function saveAdminSessions(sessions) {
  localStorage.setItem(CHAT_SESSIONS_KEY, JSON.stringify(sessions));
}

function getActiveSessionId() {
  return localStorage.getItem(ACTIVE_SESSION_KEY) || "";
}

function setActiveSessionId(id) {
  if (id) localStorage.setItem(ACTIVE_SESSION_KEY, id);
  else localStorage.removeItem(ACTIVE_SESSION_KEY);
}

function getActiveMessages() {
  const id = getActiveSessionId();
  if (!id) return [];
  const session = getAdminSessions().find((s) => s.id === id);
  return session ? session.messages : [];
}

function pushMessageToActiveSession(msg) {
  const sessions = getAdminSessions();
  let id = getActiveSessionId();
  let session = sessions.find((s) => s.id === id);
```

```
      if (!session) {
        id = "chat_" + Date.now();
        session = {
          id,
          title: msg.role === "user" ? msg.text.slice(0, 48) : "Percakapan Baru",
          updatedAt: new Date().toISOString(),
          messages: [],
        };
        sessions.unshift(session);
        setActiveSessionId(id);
      }

      session.messages.push(msg);
      session.updatedAt = new Date().toISOString();
      if (msg.role === "user" && session.messages.length === 1) {
        session.title = msg.text.length > 48 ? msg.text.slice(0, 48) + "..." : msg.text;
      }

      // Pindahkan sesi aktif ke paling atas
      const reordered = [session, ...sessions.filter((s) => s.id !== id)];
      saveAdminSessions(reordered);
    }

    function appendAdminMessageEl(item) {
      const wrapper = document.createElement("div");
      if (item.role === "user") {
        wrapper.className = "flex justify-end";
        wrapper.innerHTML = `
          <div class="bg-zinc-900 text-white rounded-2xl rounded-br-md px-3.5 py-2.5
max-w-[82%] shadow-xs">
            <p class="text-xs leading-relaxed whitespace-pre-wrap break-words [overflow-
wrap:anywhere]">${escapeHtml(item.text)}</p>
          </div>
        `;
      } else {
        wrapper.className = "flex justify-start items-start gap-2";
        let sourcesHtml = "";
        if (item.sources && item.sources.length > 0) {
          const chips = item.sources.map((s) => `
            <span class="inline-flex items-center gap-1 text-[11px] bg-zinc-50 text-
zinc-700 px-2 py-1 rounded-lg border border-zinc-200/80">
              <span class="font-medium truncate max-w-
[180px]">${escapeHtml(s.title)}</span>
              <span class="text-[10px] font-semibold text-zinc-400">${Math.round(s.score
* 100)}%</span>
            </span>
          `).join("");
          sourcesHtml = `
            <div class="mt-2.5 pt-2 border-t border-zinc-100">
              <div class="text-[10px] font-semibold text-zinc-400 uppercase tracking-
wider mb-1.5">Sumber</div>
              <div class="flex flex-wrap gap-1">${chips}</div>
            </div>
          `;
        }
        wrapper.innerHTML = `
          <div class="w-6 h-6 rounded-lg bg-indigo-50 border border-indigo-100 text-
indigo-600 flex items-center justify-center shrink-0 mt-0.5">
            <svg class="w-3 h-3" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
```

```
              <path stroke-linecap="round" stroke-linejoin="round" d="M8 10h.01M12
10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2 2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5
5v-5z" />
            </svg>
          </div>
          <div class="bg-white border border-zinc-200/80 rounded-2xl rounded-tl-md p-3.5
max-w-[calc(100%-2.25rem)] sm:max-w-[85%] min-w-0 shadow-[0_1px_2px_rgba(0,0,0,0.02)]">
            <div class="text-xs leading-relaxed text-zinc-800 break-words [overflow-
wrap:anywhere]">${formatMarkdown(item.text)}</div>
            ${sourcesHtml}
          </div>
        `;
      }
      adminChatBox.appendChild(wrapper);
    }

    function renderAdminChatHistory() {
      const history = getActiveMessages();
      if (history.length === 0) {
        adminChatBox.innerHTML = `
          <div id="adminChatEmpty" class="bg-white border border-zinc-200/80 rounded-xl
p-5 text-center my-6">
            <p class="text-xs font-medium text-zinc-700">Mulai percakapan baru</p>
            <p class="text-[11px] text-zinc-400 mt-1">Setiap sesi percakapan otomatis
tersimpan di menu Riwayat.</p>
          </div>
        `;
      } else {
        adminChatBox.innerHTML = "";
        history.forEach(appendAdminMessageEl);
        adminChatBox.scrollTop = adminChatBox.scrollHeight;
      }
    }

    function formatSessionTime(iso) {
      try {
        return new Date(iso).toLocaleString("id-ID", {
          day: "2-digit", month: "short", hour: "2-digit", minute: "2-digit"
        });
      } catch {
        return "";
      }
    }

    function renderAdminSessionsList() {
      const sessions = getAdminSessions();
      const activeId = getActiveSessionId();

      if (sessions.length === 0) {
        adminHistoryPanel.innerHTML = `
          <div class="bg-white border border-zinc-200/80 rounded-xl p-6 text-center my-
6">
            <p class="text-xs font-medium text-zinc-600">Belum ada riwayat
percakapan</p>
            <p class="text-[11px] text-zinc-400 mt-1">Mulai chat untuk menyimpan riwayat
secara otomatis.</p>
          </div>
        `;
        return;
      }

      adminHistoryPanel.innerHTML = `
```

```
        <div class="flex items-center justify-between mb-2">
          <span class="text-xs font-semibold text-zinc-700">Riwayat Percakapan
(${sessions.length})</span>
          <button type="button" onclick="clearAllAdminSessions()" class="text-[11px]
font-medium text-red-600 hover:underline cursor-pointer">
            Hapus Semua
          </button>
        </div>
      ` + sessions.map((s) => `
        <div class="bg-white border ${s.id === activeId ? "border-indigo-300 ring-1
ring-indigo-500/20" : "border-zinc-200/80"} rounded-xl p-3 flex items-center justify-
between gap-3 hover:border-indigo-200 transition">
          <button type="button" onclick="openAdminSession('${s.id}')" class="flex-1
text-left min-w-0 cursor-pointer">
            <div class="text-xs font-medium text-zinc-900
truncate">${escapeHtml(s.title)}</div>
            <div class="text-[11px] text-zinc-400 mt-0.5">${s.messages.length} pesan ·
${formatSessionTime(s.updatedAt)}</div>
          </button>
          <button type="button" onclick="deleteAdminSession('${s.id}')" class="text-
[11px] text-zinc-400 hover:text-red-600 px-2 py-1 rounded-lg hover:bg-red-50 transition
cursor-pointer shrink-0">
            Hapus
          </button>
        </div>
      `).join("");
    }

    window.toggleAdminHistoryView = function (forceState) {
      showingHistory = typeof forceState === "boolean" ? forceState : !showingHistory;
      if (showingHistory) {
        renderAdminSessionsList();
        adminHistoryPanel.classList.remove("hidden");
        adminChatBox.classList.add("hidden");
        adminChatFooter.classList.add("hidden");
        toggleHistoryBtn.textContent = "Kembali ke Chat";
      } else {
        adminHistoryPanel.classList.add("hidden");
        adminChatBox.classList.remove("hidden");
        adminChatFooter.classList.remove("hidden");
        toggleHistoryBtn.textContent = "Riwayat";
        renderAdminChatHistory();
      }
    };

    window.startNewAdminChat = function () {
      setActiveSessionId("");
      toggleAdminHistoryView(false);
      setTimeout(() => adminQuestionInput.focus(), 50);
    };

    window.openAdminSession = function (id) {
      setActiveSessionId(id);
      toggleAdminHistoryView(false);
    };

    window.deleteAdminSession = function (id) {
      const updated = getAdminSessions().filter((s) => s.id !== id);
      saveAdminSessions(updated);
      if (getActiveSessionId() === id) setActiveSessionId("");
      renderAdminSessionsList();
    };
```

```
    window.clearAllAdminSessions = function () {
      localStorage.removeItem(CHAT_SESSIONS_KEY);
      setActiveSessionId("");
      renderAdminSessionsList();
      showToast("Semua riwayat chat dihapus");
    };

    window.openChatModal = function () {
      toggleAdminHistoryView(false);
      document.getElementById("chatModal").classList.remove("hidden");
      setTimeout(() => adminQuestionInput.focus(), 50);
    };

    window.closeChatModal = function () {
      document.getElementById("chatModal").classList.add("hidden");
    };

    document.getElementById("chatModal").addEventListener("click", (e) => {
      if (e.target.id === "chatModal") closeChatModal();
    });

    adminChatForm.addEventListener("submit", async (e) => {
      e.preventDefault();
      const question = adminQuestionInput.value.trim();
      if (!question) return;

      const emptyState = document.getElementById("adminChatEmpty");
      if (emptyState) emptyState.remove();

      const userMsg = { role: "user", text: question };
      pushMessageToActiveSession(userMsg);
      appendAdminMessageEl(userMsg);

      adminQuestionInput.value = "";
      adminSendBtn.disabled = true;

      const loadingEl = document.createElement("div");
      loadingEl.id = "adminChatLoading";
      loadingEl.className = "flex justify-start";
      loadingEl.innerHTML = `
        <div class="bg-white border border-zinc-200/80 rounded-2xl rounded-bl-md px-3.5
py-2.5 shadow-xs flex items-center gap-2 text-xs text-zinc-500">
          <span class="inline-flex gap-1">
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce"></span>
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce
[animation-delay:0.15s]"></span>
            <span class="w-1.5 h-1.5 bg-indigo-500 rounded-full animate-bounce
[animation-delay:0.3s]"></span>
          </span>
          <span>Menyusun jawaban...</span>
        </div>
      `;
      adminChatBox.appendChild(loadingEl);
      adminChatBox.scrollTop = adminChatBox.scrollHeight;

      try {
        const res = await fetch("/api/chat", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ question, role: "mahasiswa" }),
        });
```

```
        const data = await res.json();
        loadingEl.remove();

        const botMsg = res.ok
          ? { role: "bot", text: data.answer, sources: data.sources || [] }
          : { role: "bot", text: data.detail || "Terjadi kesalahan pada server.",
sources: [] };

        pushMessageToActiveSession(botMsg);
        appendAdminMessageEl(botMsg);
      } catch {
        loadingEl.remove();
        const errMsg = { role: "bot", text: "Gagal terhubung ke server backend.",
sources: [] };
        pushMessageToActiveSession(errMsg);
        appendAdminMessageEl(errMsg);
      } finally {
        adminChatBox.scrollTop = adminChatBox.scrollHeight;
        adminSendBtn.disabled = false;
        adminQuestionInput.focus();
      }
    });

    window.toggleDocument = async function (id) {
      const res = await fetch(`/api/documents/${id}/toggle`, {
        method: "PATCH",
        headers: { Authorization: `Bearer ${getToken()}` },
      });
      if (res.ok) {
        loadDocuments();
      }
    };

    window.deleteDocument = async function (id, title) {
      if (!confirm(`Hapus dokumen '${title}'?`)) return;
      const res = await fetch(`/api/documents/${id}`, {
        method: "DELETE",
        headers: { Authorization: `Bearer ${getToken()}` },
      });
      if (res.ok) {
        showToast(`Dokumen '${title}' dihapus`);
        loadDocuments();
      }
    };

    window.openQuotaModal = function () {
      document.getElementById("quotaModal").classList.remove("hidden");
      loadGroqQuota();
    };

    window.closeQuotaModal = function () {
      document.getElementById("quotaModal").classList.add("hidden");
    };

    document.getElementById("quotaModal").addEventListener("click", (e) => {
      if (e.target.id === "quotaModal") closeQuotaModal();
    });

    window.loadGroqQuota = async function () {
      const contentEl = document.getElementById("quotaContent");
      const infoEl = document.getElementById("quotaModelInfo");
```

```
      contentEl.innerHTML = '<div class="text-center text-xs text-zinc-400 py-8">Memuat
data kuota...</div>';

      try {
        const res = await fetch("/api/admin/groq-quota", {
          headers: { Authorization: `Bearer ${getToken()}` },
        });
        const data = await res.json();
        if (!res.ok) {
          contentEl.innerHTML = `<div class="text-center text-xs text-red-600 py-
6">${escapeHtml(data.detail || "Gagal memuat kuota.")}</div>`;
          return;
        }

        const activeModel = (data.latest && data.latest.model) || data.configured_model;
        infoEl.textContent = `Model: ${activeModel} • ${data.has_api_key ? "API Key
Aktif" : "API Key Belum Diatur"}`;

        const latest = data.latest;
        const reqLimit = latest && latest.limit_requests ? latest.limit_requests : 0;
        const reqRem = latest && latest.remaining_requests != null ?
latest.remaining_requests : 0;
        const reqPct = reqLimit > 0 ? Math.min(100, Math.round((reqRem / reqLimit) *
100)) : 0;

        const tokLimit = latest && latest.limit_tokens ? latest.limit_tokens : 0;
        const tokRem = latest && latest.remaining_tokens != null ?
latest.remaining_tokens : 0;
        const tokPct = tokLimit > 0 ? Math.min(100, Math.round((tokRem / tokLimit) *
100)) : 0;

        const rateLimitHtml = latest
          ? `
            <div class="grid grid-cols-1 sm:grid-cols-2 gap-3">
              <div class="bg-white border border-zinc-200/80 rounded-xl p-4">
                <div class="flex items-center justify-between text-xs font-medium text-
zinc-600">
                  <span>Sisa Request (RPD)</span>
                  <span class="text-zinc-900 font-semibold">${reqRem.toLocaleString("id-
ID")} / ${reqLimit.toLocaleString("id-ID")}</span>
                </div>
                <div class="w-full h-2 bg-zinc-100 rounded-full overflow-hidden mt-2.5">
                  <div class="h-full ${reqPct > 25 ? "bg-indigo-600" : "bg-amber-500"}
rounded-full transition-all" style="width: ${reqPct}%"></div>
                </div>
                <div class="flex items-center justify-between text-[11px] text-zinc-400
mt-2">
                  <span>${reqPct}% tersisa</span>
                  <span>Reset: ${escapeHtml(latest.reset_requests_label ||
latest.reset_requests || "-")}</span>
                </div>
              </div>

              <div class="bg-white border border-zinc-200/80 rounded-xl p-4">
                <div class="flex items-center justify-between text-xs font-medium text-
zinc-600">
                  <span>Sisa Token (TPM)</span>
                  <span class="text-zinc-900 font-semibold">${tokRem.toLocaleString("id-
ID")} / ${tokLimit.toLocaleString("id-ID")}</span>
                </div>
                <div class="w-full h-2 bg-zinc-100 rounded-full overflow-hidden mt-2.5">
```

```
                  <div class="h-full ${tokPct > 25 ? "bg-emerald-500" : "bg-amber-500"}
rounded-full transition-all" style="width: ${tokPct}%"></div>
                </div>
                <div class="flex items-center justify-between text-[11px] text-zinc-400
mt-2">
                  <span>${tokPct}% tersisa</span>
                  <span>Reset: ${escapeHtml(latest.reset_tokens_label ||
latest.reset_tokens || "-")}</span>
                </div>
              </div>
            </div>
          `
          : `
            <div class="bg-amber-50 border border-amber-200/80 rounded-xl p-4 text-xs
text-amber-800">
              Belum ada request ke Groq API yang tercatat. Kirim minimal 1 pertanyaan di
chatbot untuk melihat sisa kuota real-time dari header Groq.
            </div>
          `;

        const statsHtml = `
          <div class="grid grid-cols-3 gap-3">
            <div class="bg-white border border-zinc-200/80 rounded-xl p-3.5">
              <div class="text-[11px] text-zinc-500">Request Hari Ini</div>
              <div class="text-lg font-semibold text-zinc-900 mt-
0.5">${(data.today.requests_today || 0).toLocaleString("id-ID")}</div>
            </div>
            <div class="bg-white border border-zinc-200/80 rounded-xl p-3.5">
              <div class="text-[11px] text-zinc-500">Token Hari Ini</div>
              <div class="text-lg font-semibold text-zinc-900 mt-
0.5">${(data.today.total_tokens_today || 0).toLocaleString("id-ID")}</div>
            </div>
            <div class="bg-white border border-zinc-200/80 rounded-xl p-3.5">
              <div class="text-[11px] text-zinc-500">Total Request</div>
              <div class="text-lg font-semibold text-zinc-900 mt-
0.5">${(data.total.total_requests || 0).toLocaleString("id-ID")}</div>
            </div>
          </div>
        `;

        const logs = data.recent_logs || [];
        const logsHtml = logs.length === 0
          ? `<div class="text-center text-xs text-zinc-400 py-5">Belum ada riwayat
pemanggilan Groq API.</div>`
          : `
            <div class="bg-white border border-zinc-200/80 rounded-xl overflow-hidden">
              <div class="px-4 py-2.5 border-b border-zinc-100 text-xs font-semibold
text-zinc-800">10 Request Terakhir</div>
              <div class="overflow-x-auto">
                <table class="w-full text-left border-collapse text-xs">
                  <thead class="bg-zinc-50 border-b border-zinc-100 text-[11px] text-
zinc-500">
                    <tr>
                      <th class="px-3.5 py-2 font-medium">Waktu</th>
                      <th class="px-3.5 py-2 font-medium">Model</th>
                      <th class="px-3.5 py-2 font-medium">Token</th>
                      <th class="px-3.5 py-2 font-medium">Status</th>
                    </tr>
                  </thead>
                  <tbody class="divide-y divide-zinc-100">
                    ${logs.map((l) => {
```

```
                      const dt = new Date(l.created_at).toLocaleString("id-ID", { day:
"2-digit", month: "short", hour: "2-digit", minute: "2-digit" });
                      const badge = l.status === "success"
                        ? '<span class="text-[10px] bg-emerald-50 text-emerald-700
border border-emerald-200/70 px-1.5 py-0.5 rounded font-medium">Sukses</span>'
                        : '<span class="text-[10px] bg-red-50 text-red-700 border
border-red-200/70 px-1.5 py-0.5 rounded font-medium">Error</span>';
                      return `
                        <tr>
                          <td class="px-3.5 py-2 text-zinc-500 whitespace-
nowrap">${escapeHtml(dt)}</td>
                          <td class="px-3.5 py-2 text-zinc-800 font-mono text-
[11px]">${escapeHtml(l.model)}</td>
                          <td class="px-3.5 py-2 text-zinc-700 whitespace-
nowrap">${(l.total_tokens || 0).toLocaleString("id-ID")}</td>
                          <td class="px-3.5 py-2 whitespace-nowrap">${badge}</td>
                        </tr>
                      `;
                    }).join("")}
                  </tbody>
                </table>
              </div>
            </div>
          `;

        contentEl.innerHTML = rateLimitHtml + statsHtml + logsHtml;
      } catch {
        contentEl.innerHTML = '<div class="text-center text-xs text-red-600 py-6">Gagal
terhubung ke server.</div>';
      }
    };

    checkAuth();
  </script>
</body>
</html>
```

## Langkah 13: Menjalankan Server dan Pengujian

Jalankan server FastAPI menggunakan Uvicorn dari folder utama project:

**Terminal**

```
uvicorn app.main:app --reload
```

Setelah server berjalan, lakukan pengujian berikut melalui browser:

1. Buka `http://127.0.0.1:8000` untuk mengakses halaman Chatbot Mahasiswa. Klik salah satu pertanyaan rekomendasi atau ketik pertanyaan tentang SSC maupun TAK, lalu periksa jawaban dan klik kartu **Sumber Referensi** untuk melihat popup isi dokumen.
2. Buka `http://127.0.0.1:8000/admin` dan login menggunakan akun default (`admin` / `admin123`).
3. Uji fitur **Upload Dokumen**, klik **Lihat Chunks**, coba tombol **Nonaktifkan / Aktifkan**, serta uji **Buka Chatbot** dan **Kuota Groq API** pada sidebar.

## 3. Troubleshooting

| Masalah | Penyebab Umum | Solusi |
|---|---|---|
| 'python' atau 'pip' is not recognized (Masalah PATH) | Saat instalasi Python di Windows, opsi Add Python to PATH tidak dicentang. | Gunakan launcher py (misal: py -m venv venv atau py -m pip install -r requirements.txt), atau tambahkan folder Python & Scripts ke Environment Variables PATH lalu buka ulang terminal. |
| Aktivasi venv gagal di PowerShell (ExecutionPolicy) | Kebijakan keamanan PowerShell memblokir eksekusi script Activate.ps1. | Jalankan Set-ExecutionPolicy -Scope CurrentUser RemoteSigned di PowerShell, atau gunakan Command Prompt (cmd) untuk menjalankan venv\Scripts\activate. |
| ModuleNotFoundError: No module named 'app' | Perintah uvicorn dijalankan dari luar folder root modul2ai, atau file app/__init__.py belum dibuat. | Pastikan terminal berada tepat di dalam folder modul2ai (cek dengan cd / pwd) dan pastikan file app/__init__.py tersedia. |
| ModuleNotFoundError: No module named 'fastapi' / 'groq' | Dependensi terpasang di Python global, bukan di virtual environment (venv) yang sedang aktif. | Pastikan tanda (venv) muncul di awal baris terminal, lalu jalankan ulang pip install -r requirements.txt. |
| Port 8000 sudah digunakan (Address already in use) | Masih ada proses Uvicorn atau aplikasi lain yang memakai port 8000. | Tutup terminal server sebelumnya atau jalankan di port berbeda: uvicorn app.main:app --reload --port 8001. |
| Pesan 'GROQ_API_KEY belum dikonfigurasi' | File .env belum diisi API Key atau server belum di-restart setelah mengubah .env. | Isi GROQ_API_KEY di file .env dengan key valid dari console.groq.com, simpan, lalu restart Uvicorn. |
| Error 429 rate_limit_exceeded (OTPM) / model_not_found | Limit Output Tokens Per Minute (OTPM) terlampaui atau nama model Groq sudah berubah. | Pastikan max_tokens di app/rag.py bernilai <= 700, atau perbarui GROQ_MODEL di .env sesuai daftar model aktif di console.groq.com. |
| Perubahan file di folder dataset/ tidak muncul | Database modul2ai.db sudah terisi sebelumnya sehingga seeding otomatis saat startup dilewati. | Hapus dokumen lama dari panel admin lalu upload ulang, atau hapus file modul2ai.db dan restart Uvicorn. |
| Tampilan web tidak berubah setelah edit HTML/JS | Browser masih menggunakan cache file statis versi sebelumnya. | Lakukan hard refresh pada browser menggunakan Ctrl + F5 (atau Cmd + Shift + R di macOS). |

## 4. Tugas

> Tugas dikerjakan secara **individu** selama satu minggu setelah praktikum. Kumpulkan laporan menggunakan **Template Laporan Praktikum Individu** melalui LMS.

Sebelum mengerjakan tugas *deployment*, pelajari terlebih dahulu panduan langkah demi langkah pada modul berikut:

**Download Modul Deployment FastAPI ke Railway**

Setelah mempelajari modul tersebut, kembangkan aplikasi `modul2ai` dengan menyelesaikan ketentuan berikut:

4. **Tambahkan Manajemen Role Multi-Pengguna:** Aktifkan kembali fitur pengelolaan *role* di panel admin agar admin dapat menambah/menghapus *role* baru (contoh: `mahasiswa` dan `dosen`), serta memilih *role* mana saja yang berhak mengakses setiap dokumen saat proses upload.
5. **Pemisahan Akses Dokumen Sensitif:** Unggah minimal satu dokumen khusus yang hanya dapat diakses oleh role `dosen` (misalnya panduan internal dosen atau kebijakan penilaian). Pastikan saat chatbot ditanya menggunakan role `mahasiswa`, informasi dari dokumen tersebut **tidak muncul**, sedangkan saat menggunakan role `dosen`, chatbot dapat menjawab akurat beserta sumber referensinya.
6. **Deployment ke Cloud:** Lakukan *deployment* aplikasi backend FastAPI beserta antarmuka webnya ke Railway menggunakan panduan pada link di atas, lalu lampirkan link aplikasi aktif dan tangkapan layar pengujiannya pada laporan praktikum.